

DATA STRUCTURES AND ALGORITHMS

23CSE203 LAB MANUAL

SUBMITTED BY	
NAME	M. RISHIVARDHAN
ROLL NO	AV.SC.U4CSE25125
YEAR/SEM/SECTION	2nd YEAR/SEM-3/CSE-B
CREDITS	05

SUBMITTED TO DR. RAJ KUMAR BATCHU	
DESIGNATION	ASST. PROFESSOR (SI. GRADE)
DEPARTMENT	SCHOOL OF COMPUTING

I N D E X

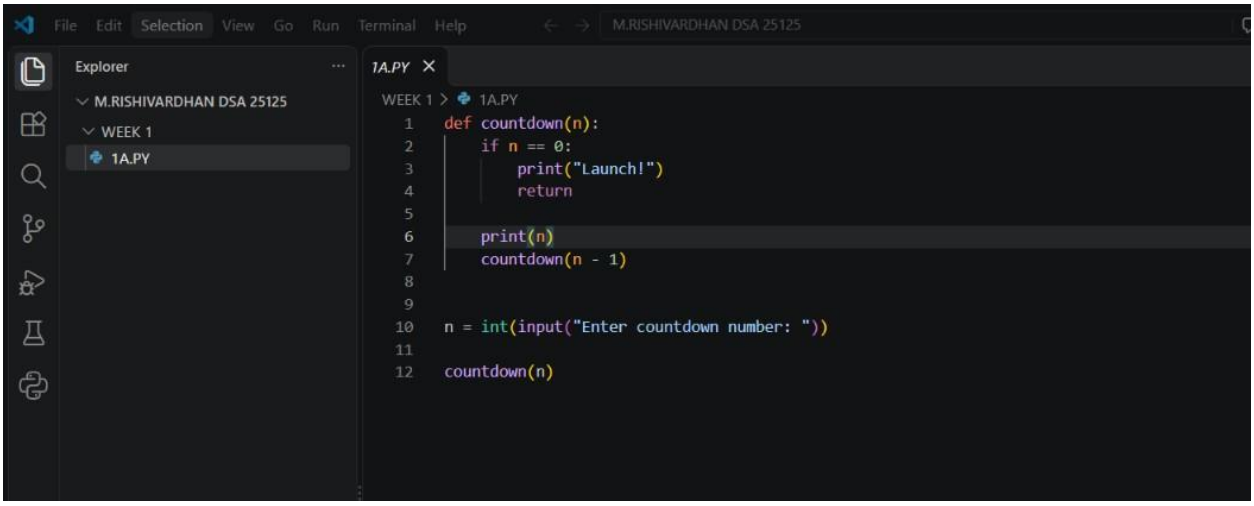
S.NO	T I T L E	Page no	Signature
1	Week1 (a). A rocket launch system displays a countdown before launch. Instead of loop implement countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!". (b) Banks calculate the growth of an investment using compound interest. The formulae require calculating powers, which can be implemented recursively. Write a recursive program to calculate P^N where: P=Principal Growth, N=number of years. (c) An HR department stores ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in the list. (d) In logistics company, develop a recursive program to calculate the factorial of a given number. (e) Write a recursive program to print the first n terms of the Fibonacci series	5-9	
2	Week-2 (a) Linear Search (b) Binary Search without sorting (c) Binary Search with sorted array	9-12	
3	Week-3 (a) Bubble Sort	13-15	

	(b) Insertion Sort **(c) Selection Sort**		
4	**Week 4** **(a) Write a Python program to implement Merge Sort.** **(b) Write a Python program to implement Quick Sort.**	**16-18**	
5	**Week 5 5(a). Write a Python program to implement a Singly Linked List (SLL) with the following operations:** **5(b) Write a Python program to implement a Double Linked List with basic operations.** **5(c) Write a Python program to implement a Circular Linked List with basic operations.**	**18-33**	
6	**Week 6 Write a Python program to implement Stack operations such as Push, Pop, Peek, and Display using an array and the linked list.**	**33-39**	
7	**Week 7 (a) Write a Python program to implement a Queue and its operations such as Enqueue, Dequeue, Peek, and Display with a menu-driven approach.** **Implement using array and SLL.** **(b) Write a Python Program to implement Circular Queue**	**40-52**	
8	**Write a program to implement Binary Tree using array and single linked list**	**53-56**	
9	**Write a python program to implement binary search tree and its methods**	**57-65**	
10	**Write a python program to implement AVL trees and its methods**	**66-74**	
11	**Write a program to implement hashing concept**	**74-77**	

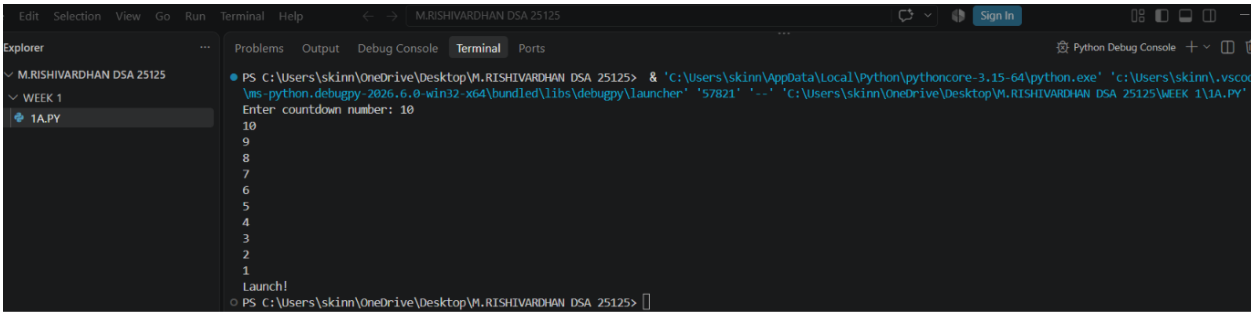
Week1

(a). A rocket launch system displays a countdown before launch. Instead of loop implement countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!".

Code:



Output:



Description:

This program displays a rocket launch countdown from n to 1 using recursion instead of loops. The function prints the current countdown number and recursively calls itself with the value decreased by 1. When the value reaches 0, the message "Launch!" is displayed, terminating the recursion.

(b) Banks calculate the growth of an investment using compound interest. The formulae require calculating powers, which can be implemented recursively. Write a recursive program to calculate P^N where: P=Principal Growth, N=number of years.

Code:

```python
def power(p, n):
    if n == 0:
        return 1
    return p * power(p, n - 1)


p = float(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))

result = power(p, n)

print("P^n =", result)
```

Output:

```
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125> & 'C:\Users\skinn\AppData\Local\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50485' '--' 'C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 1\1B.PY'
Enter principal growth factor: 1.25
Enter number of years: 3
P^n = 1.953125
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125>
```

Description:

This program calculates P^N using recursion. The function repeatedly multiplies the principal growth factor P by itself while decreasing the value of n by 1. The recursion stops when n becomes 0, since any number raised to power 0 is 1. The final result represents the growth factor of the investment over n years.

(c) An HR department stores ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in the list.

Code:

```python
def search(arr, employeeID, index):
    if index == len(arr):
        return -1

    if arr[index] == employeeID:
        return index

    return search(arr, employeeID, index + 1)


employees = [101, 205, 310, 415, 520]

employeeID = int(input("Enter Employee ID: "))

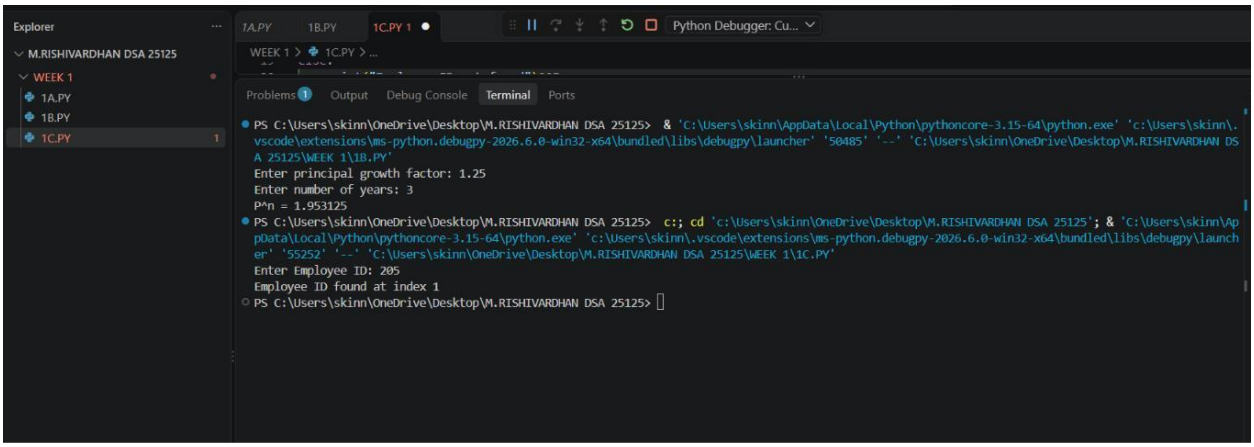
result = search(employees, employeeID, 0)

if result != -1:
    print("Employee ID found at index", result)
else:
    print("Employee ID not found")
```

OUTPUT:

B L A N K

P A G E

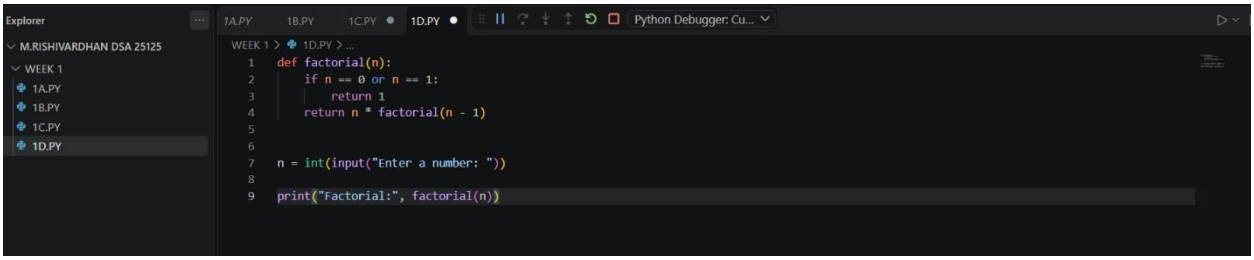


Description:

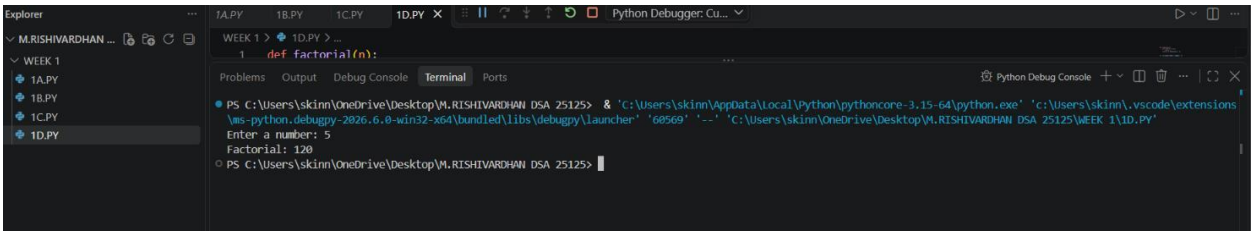
This program searches for a given employee ID in a list using recursion. The function checks each element of the list one by one and recursively moves to the next index until the employee ID is found, or the end of the list is reached. If the ID is found, its index is returned; otherwise, -1 is returned.

(d) In logistics company, develop a recursive program to calculate the factorial of a given number.

Code:



Output:



Description:

This program calculates the factorial of a given number using recursion. The function multiplies the current number by the factorial of the previous number. The recursion stops when the number becomes 0 or 1, as the factorial of both 0 and 1 is 1.

(E)

Write a recursive program to print the first n terms of the Fibonacci series

Code:



Output:



Description:

This program generates the first n terms of the Fibonacci series using recursion. Each Fibonacci term is calculated as the sum of the previous two terms. The base cases are F(0)=0 and F(1)=1. The program recursively calculates and displays each term.

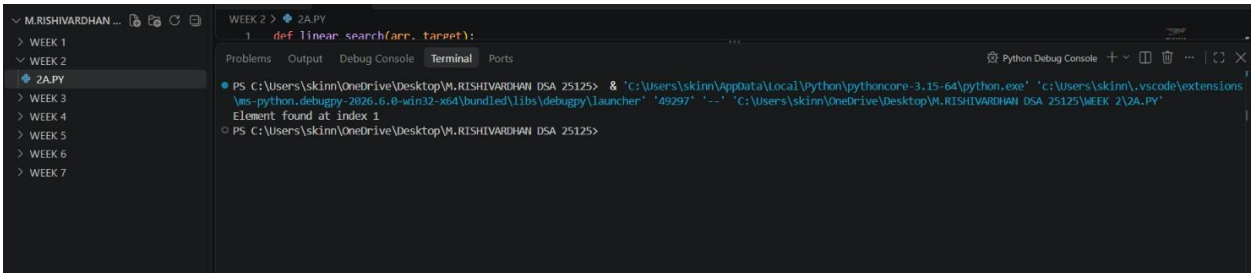
Week-2

(a). Linear Search

Code:



Output:



Description:

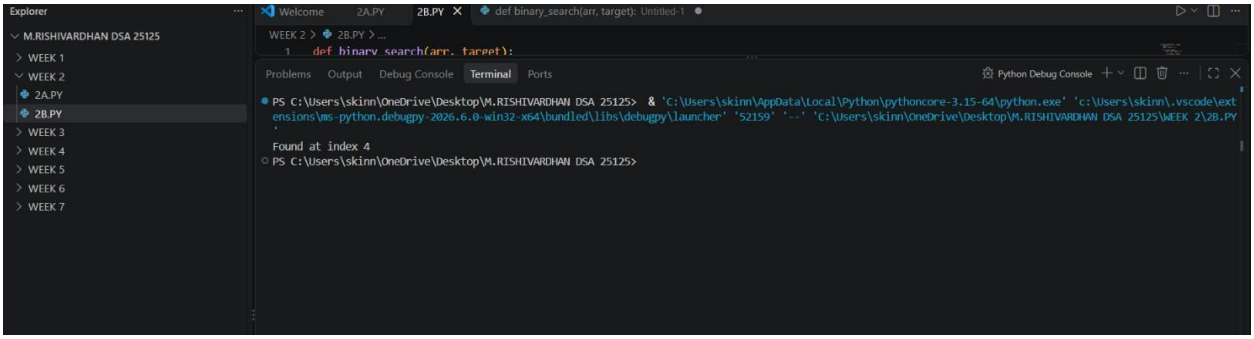
Linear Search is a searching algorithm that checks each element of a list sequentially from the beginning until the required element is found, or the end of the list is reached. It can be used on both sorted and unsorted lists. The time complexity of Linear Search is O(n).

(b) Binary Search (sorted array)

Code:



Output:

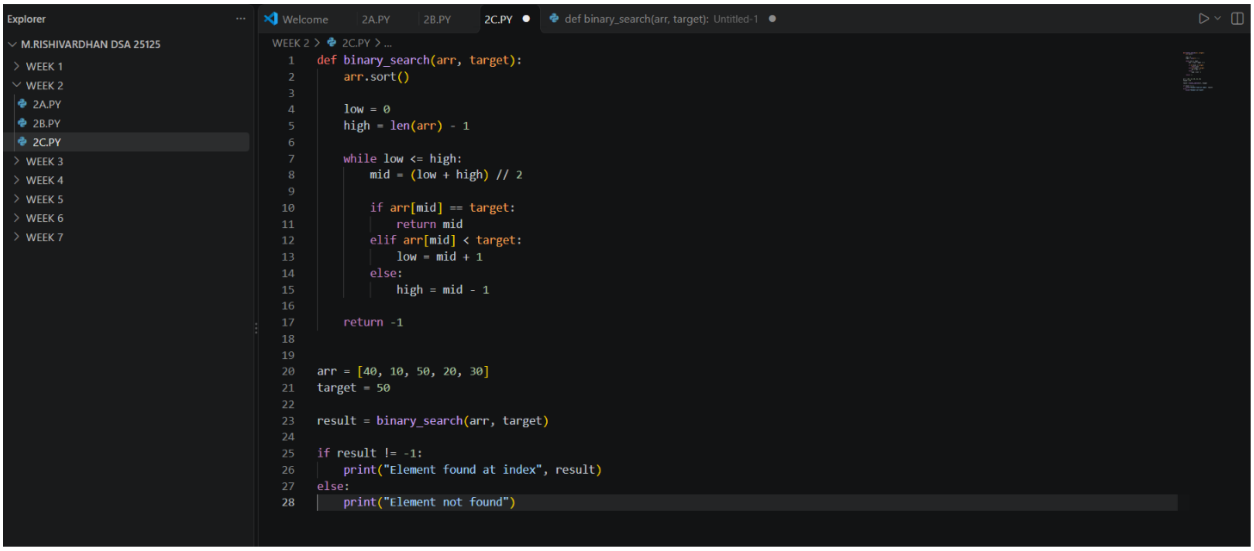


Description:

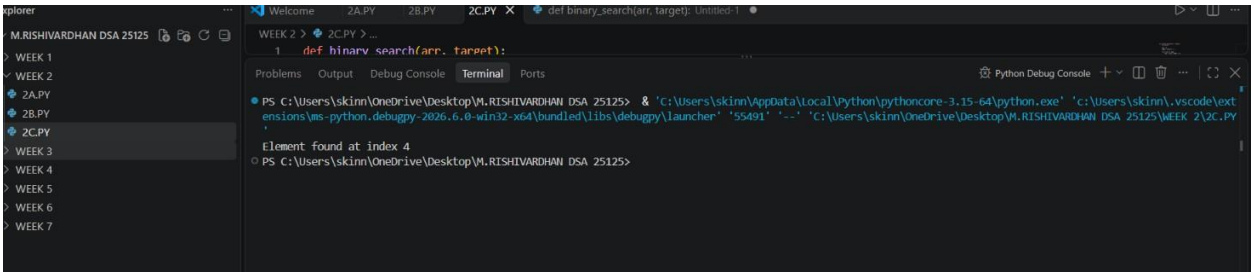
Binary Search is an efficient searching algorithm used on a sorted list. It compares the target element with the middle element of the list and eliminates half of the search space after each comparison. This process is repeated until the element is found or the search range becomes empty. The time complexity of Binary Search is O(log n).

(c) Write a Python Program to implement Binary Search(unsorted array)

Code:



Output:



DESCRIPTION:

Binary Search cannot be directly applied to an unsorted list because it requires the elements to be arranged in sorted order. Therefore, the elements must first be arranged in ascending order before applying Binary Search. After sorting, the middle element is compared with the target, and half of the search space is eliminated at each step. The overall time complexity, including sorting, is O(n log n).

Week-3

(a). Write a Python Program to implement Bubble Sort

Code:



```
M.RISHIVARDHAN DSA 25125
WEEK 1
WEEK 2
2A.PY
2B.PY
2C.PY
WEEK 3
3A.PY
WEEK 4
WEEK 5
WEEK 6
WEEK 7

WEEK 3 > 3A.PY > ...
def bubble_sort(arr):
    n = len(arr)

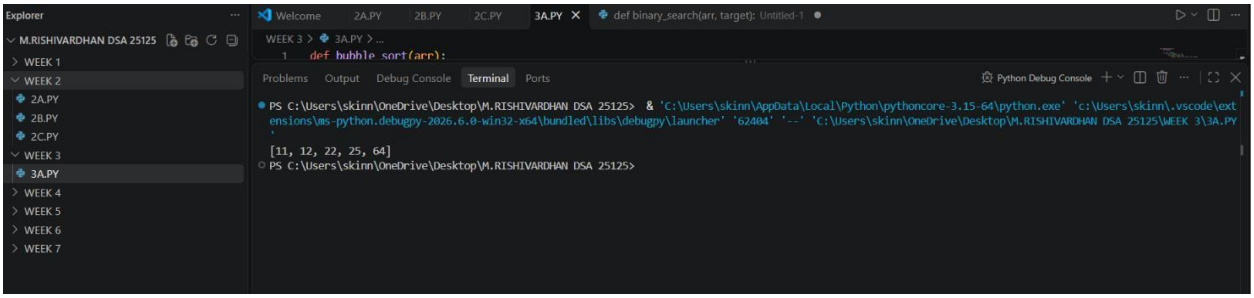
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr


arr = [64, 25, 12, 22, 11]

print(bubble_sort(arr))
```

Output:



```
Explorer
M.RISHIVARDHAN DSA 25125
WEEK 1
WEEK 2
2A.PY
2B.PY
2C.PY
WEEK 3
3A.PY
WEEK 4
WEEK 5
WEEK 6
WEEK 7

Welcome  2A.PY  2B.PY  2C.PY  3A.PY  def binary_search(arr, target): Untitled-1
WEEK 3 > 3A.PY > ...
def bubble_sort(arr):

Problems  Output  Debug Console  Terminal  Ports  Python Debug Console

PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125> & 'C:\Users\skinn\AppData\Local\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '62404' '--' 'C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 3\3A.PY'
[11, 12, 22, 25, 64]
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125>
```

Description:

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. During each pass, the largest unsorted element moves to the end of the list. The process is repeated until the complete list is sorted. The time complexity of Bubble Sort is O(n^2).

(b) Write a Python Program to implement Insertion Sort

Code:



```
Explorer
M.RISHIVARDHAN DSA 25125
WEE M.RISHIVARDHAN DSA 25125
WEEK 2
2A.PY
2B.PY
2C.PY
WEEK 3
3A.PY
3B.PY
WEEK 4
WEEK 5
WEEK 6
WEEK 7

Welcome  2A.PY  2B.PY  2C.PY  3A.PY  3B.PY  def binary_search(arr, target): Untitled-1
WEEK 3 > 3B.PY > ...
def insertion_sort(arr):
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

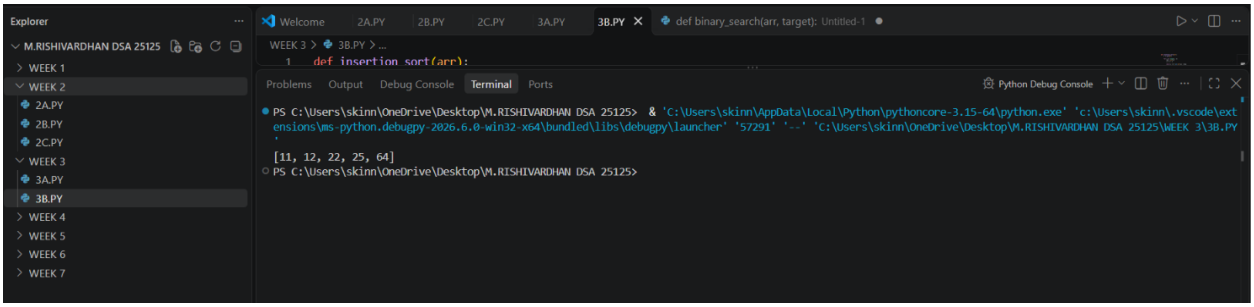
        arr[j + 1] = key

    return arr


arr = [64, 25, 12, 22, 11]

print(insertion_sort(arr))
```

Output:

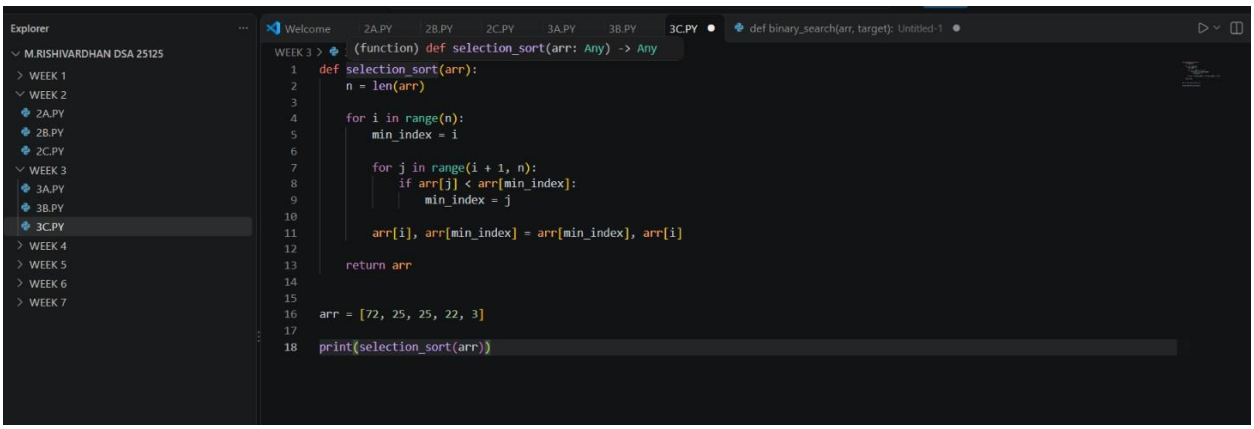


Description:

Insertion Sort is a sorting algorithm that builds the sorted list one element at a time. It takes each element from the unsorted portion and inserts it into its correct position in the sorted portion by shifting larger elements to the right. The process continues until all elements are sorted. The worst-case time complexity of Insertion Sort is O(n^2).

(c) Write a Python Program to implement Selection Sort

Code:



Output:



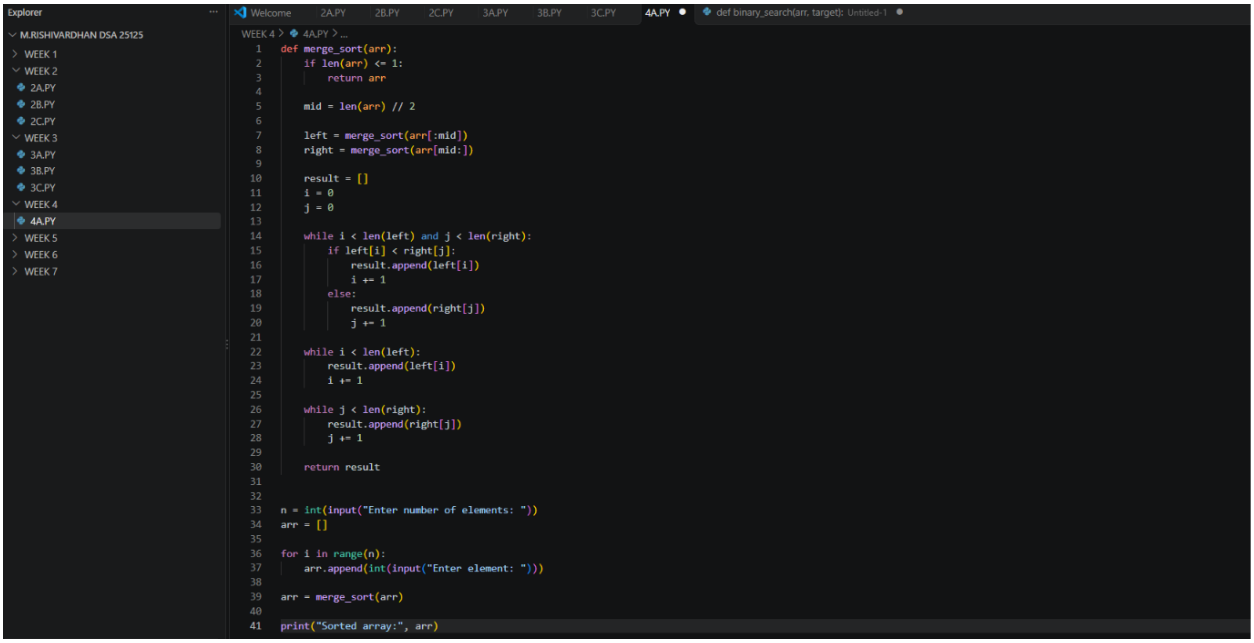
Description:

Selection Sort repeatedly finds the smallest element from the unsorted portion of the list and places it at the beginning of that portion. This process is repeated until all elements are arranged in ascending order. The time complexity of Selection Sort is O(n^2).

Week-4

4(a) Write a Python Program to implement Merge Sort

Code:



```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    while i < len(left):
        result.append(left[i])
        i += 1

    while j < len(right):
        result.append(right[j])
        j += 1

    return result

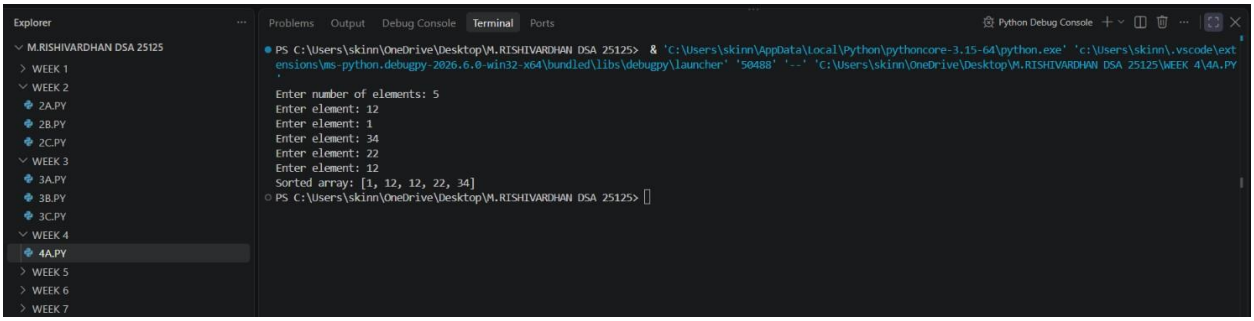

n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    arr.append(int(input("Enter element: ")))

arr = merge_sort(arr)

print("Sorted array:", arr)
```

Output:



```
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125> & 'C:\Users\skinn\AppData\Local\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50488' '--' 'C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 4\4A.PY'
Enter number of elements: 5
Enter element: 12
Enter element: 1
Enter element: 34
Enter element: 22
Enter element: 12
Sorted array: [1, 12, 12, 22, 34]
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125>
```

Description:

Merge Sort is a divide-and-conquer sorting algorithm. The list is repeatedly divided into two halves until each sub-list contains a single element, and then the sorted sub-lists are merged back together in sorted order. The time complexity of Merge Sort is O(n log n).

(b) Write a Python Program to implement Quick Sort

Code:

```python
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    left = []
    right = []

    for i in range(len(arr) - 1):
        if arr[i] < pivot:
            left.append(arr[i])
        else:
            right.append(arr[i])

    return quick_sort(left) + [pivot] + quick_sort(right)


n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input("Enter element: ")))

arr = quick_sort(arr)

print("Sorted array:", arr)
```

Output:

```
Enter number of elements: 5
Enter element: 12
Enter element: 21
Enter element: 3
Enter element: 19
Enter element: 45
Sorted array: [3, 12, 19, 21, 45]
```

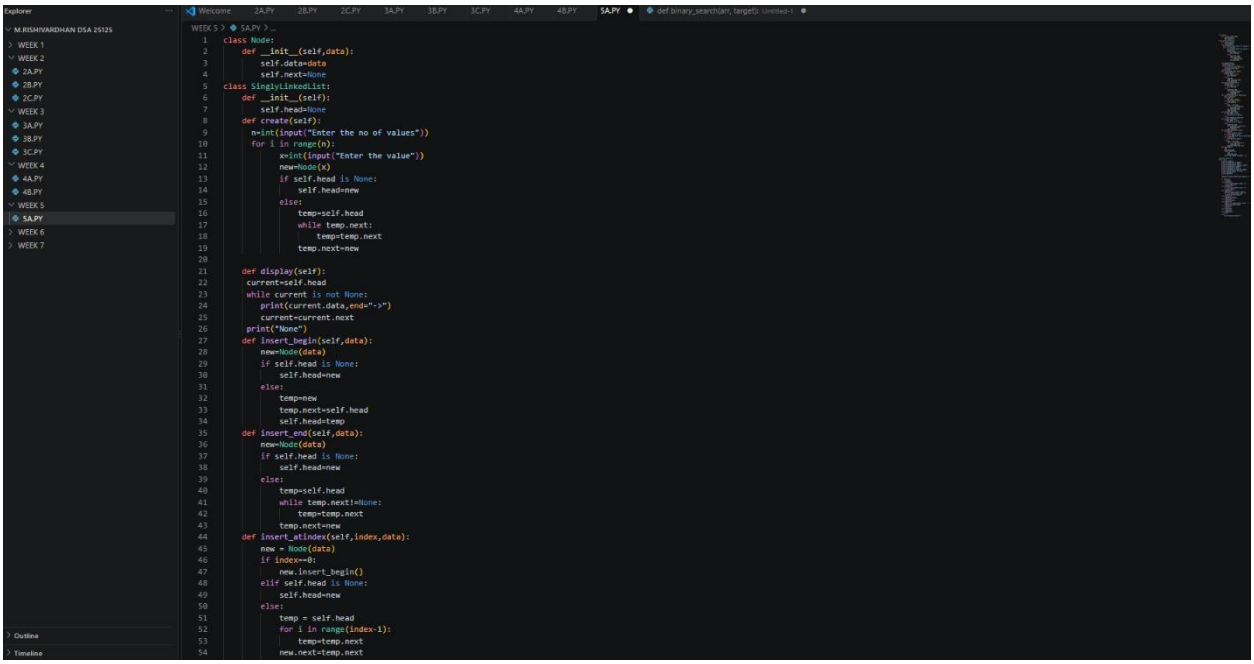
Description:

Quick Sort is a divide-and-conquer sorting algorithm. It selects a pivot element and partitions the list into two sub-lists: elements smaller than the pivot and elements greater than the pivot. The process is recursively applied to the sub-lists. The average time complexity of Quick Sort is O(n log n).

Week-5

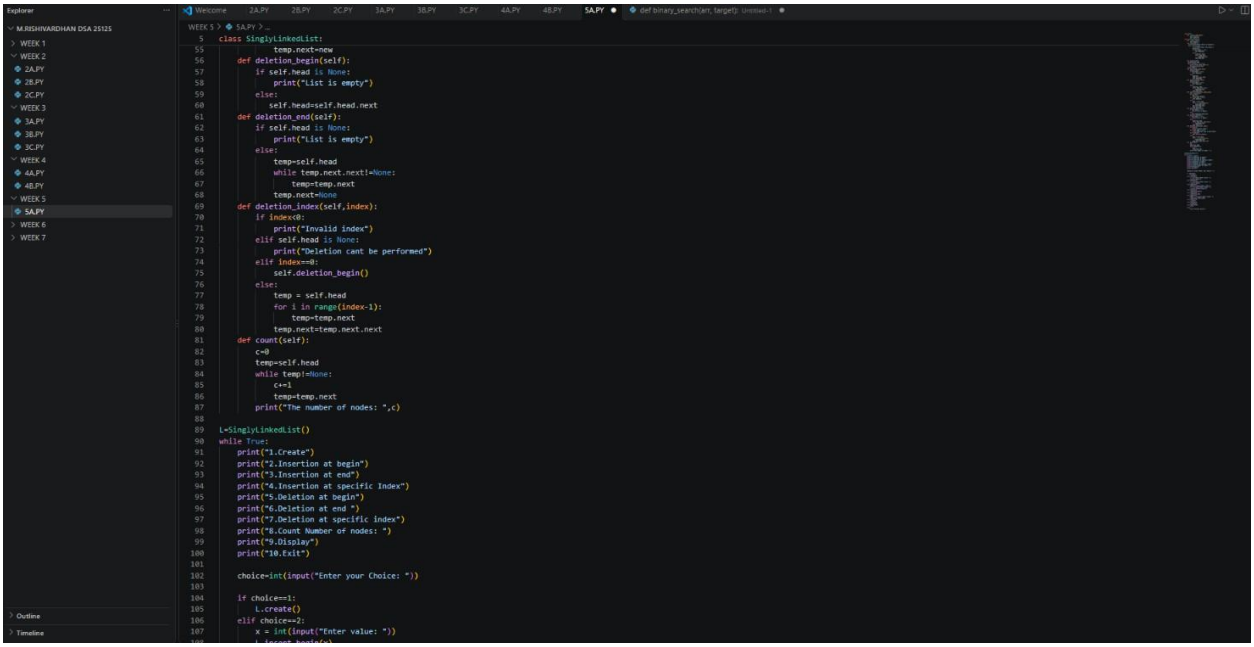
(a). Write a Python program to implement SLL and its basic operations

Code:



```
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None
class SinglyLinkedList:
    def __init__(self):
        self.head=None
    def create(self):
     n=int(input("Enter the no of values"))
     for i in range(n):
            x=int(input("Enter the value"))
            new=Node(x)
            if self.head is None:
                self.head=new
            else:
                temp=self.head
                while temp.next:
                    temp=temp.next
                temp.next=new

    def display(self):
     current=self.head
     while current is not None:
        print(current.data,end="->")
        current=current.next
     print("None")
    def insert_begin(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            temp=new
            temp.next=self.head
            self.head=temp
    def insert_end(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
        else:
            temp=self.head
            while temp.next!=None:
                temp=temp.next
            temp.next=new
    def insert_atindex(self,index,data):
        new = Node(data)
        if index==0:
            new.insert_begin()
        elif self.head is None:
            self.head=new
        else:
            temp = self.head
            for i in range(index-1):
                temp=temp.next
            new.next=temp.next
```

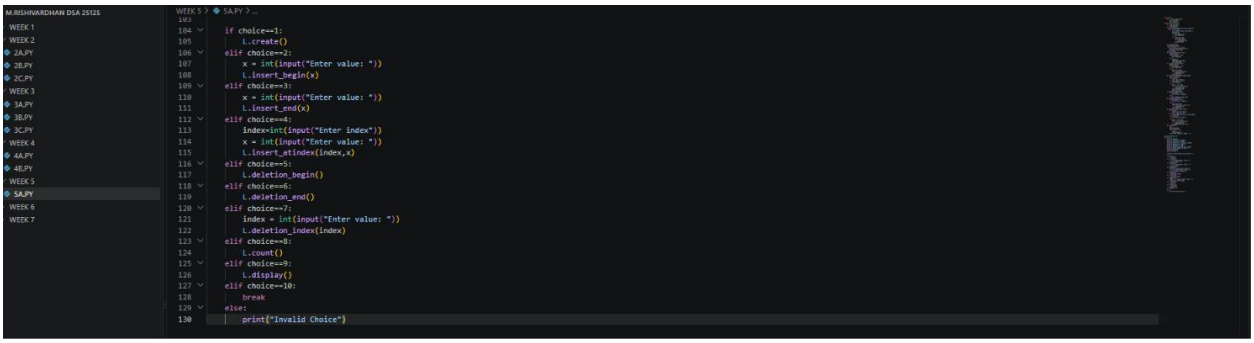


```
            temp.next=new
    def deletion_begin(self):
        if self.head is None:
            print("List is empty")
        else:
            self.head=self.head.next
    def deletion_end(self):
        if self.head is None:
            print("List is empty")
        else:
            temp=self.head
            while temp.next.next!=None:
                temp=temp.next
            temp.next=None
    def deletion_index(self,index):
        if index<0:
            print("Invalid index")
        elif self.head is None:
            print("Deletion cant be performed")
        elif index==0:
            self.deletion_begin()
        else:
            temp = self.head
            for i in range(index-1):
                temp=temp.next
            temp.next=temp.next.next
    def count(self):
        c=0
        temp=self.head
        while temp!=None:
            c+=1
            temp=temp.next
        print("The number of nodes: ",c)

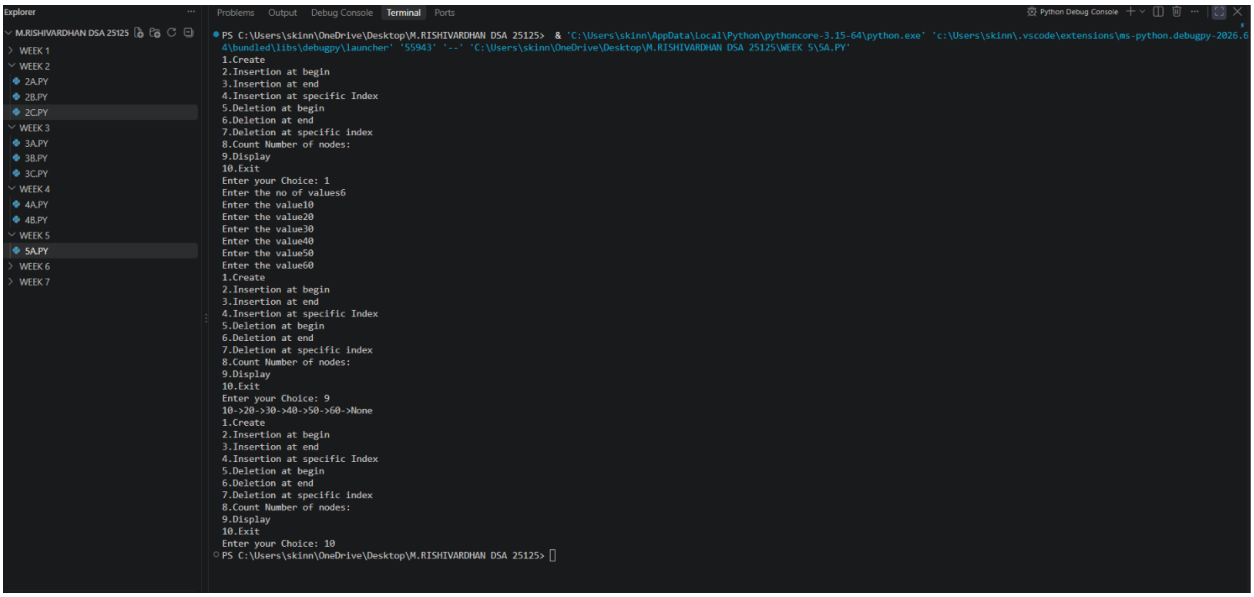
L=SinglyLinkedList()
while True:
    print("1.Create")
    print("2.Insertion at begin")
    print("3.Insertion at end")
    print("4.Insertion at specific Index")
    print("5.Deletion at begin")
    print("6.Deletion at end ")
    print("7.Deletion at specific index")
    print("8.Count Number of nodes: ")
    print("9.Display")
    print("10.Exit")

    choice=int(input("Enter your Choice: "))

    if choice==1:
        L.create()
    elif choice==2:
        x = int(input("Enter value: "))
```



Output:

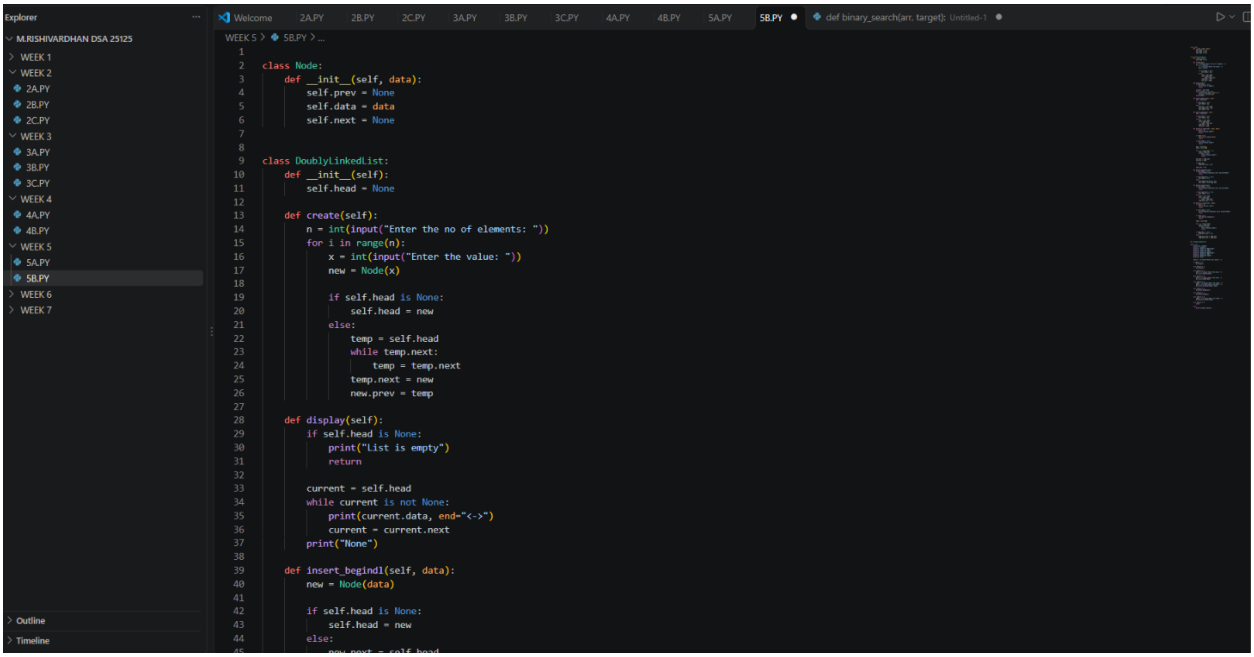


Description:

This program implements a Singly Linked List with a menu-driven approach. Each node contains data and a pointer to the next node. The program supports operations such as create, insertion at beginning/end/specific index, deletion at beginning/end/specific index, counting the number of nodes, and displaying the list.

(b) Write a Python program to implement DLL with the basic operations

Code:



```
class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter the no of elements: "))
        for i in range(n):
            x = int(input("Enter the value: "))
            new = Node(x)

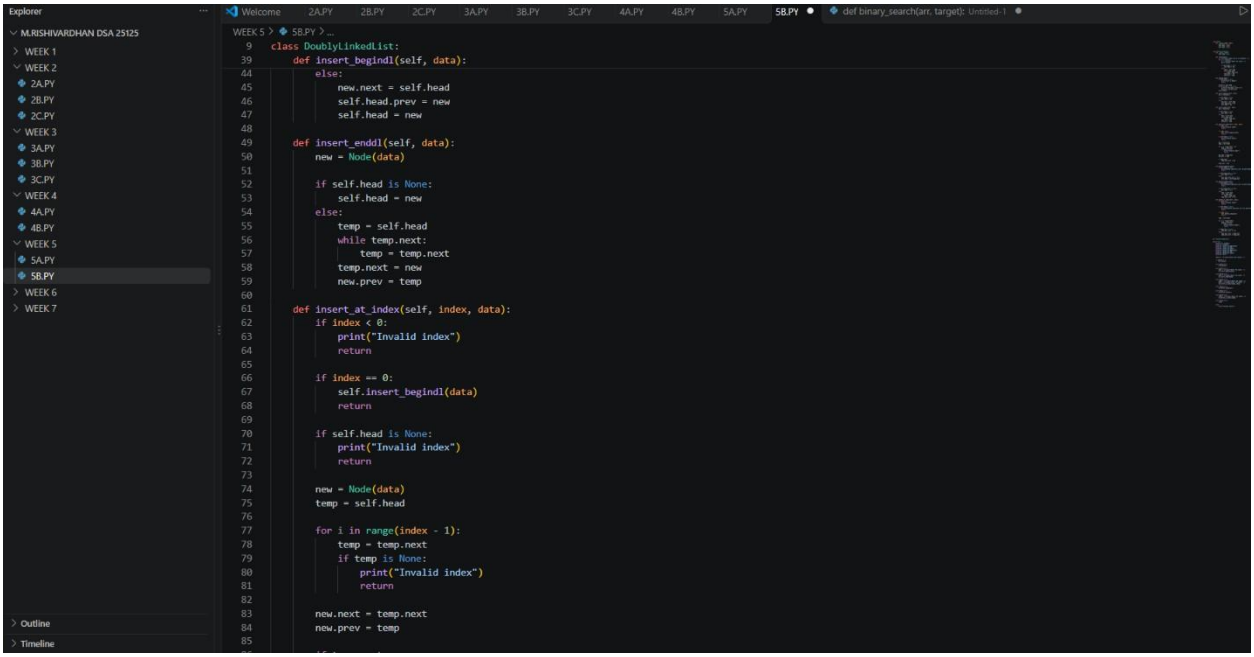
            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new
                new.prev = temp

    def display(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head
        while current is not None:
            print(current.data, end="<->")
            current = current.next
        print("None")

    def insert_begindl(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
```



```
class DoublyLinkedList:
    def insert_begindl(self, data):
            else:
                new.next = self.head
                self.head.prev = new
                self.head = new

    def insert_enddl(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new
            new.prev = temp

    def insert_at_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return

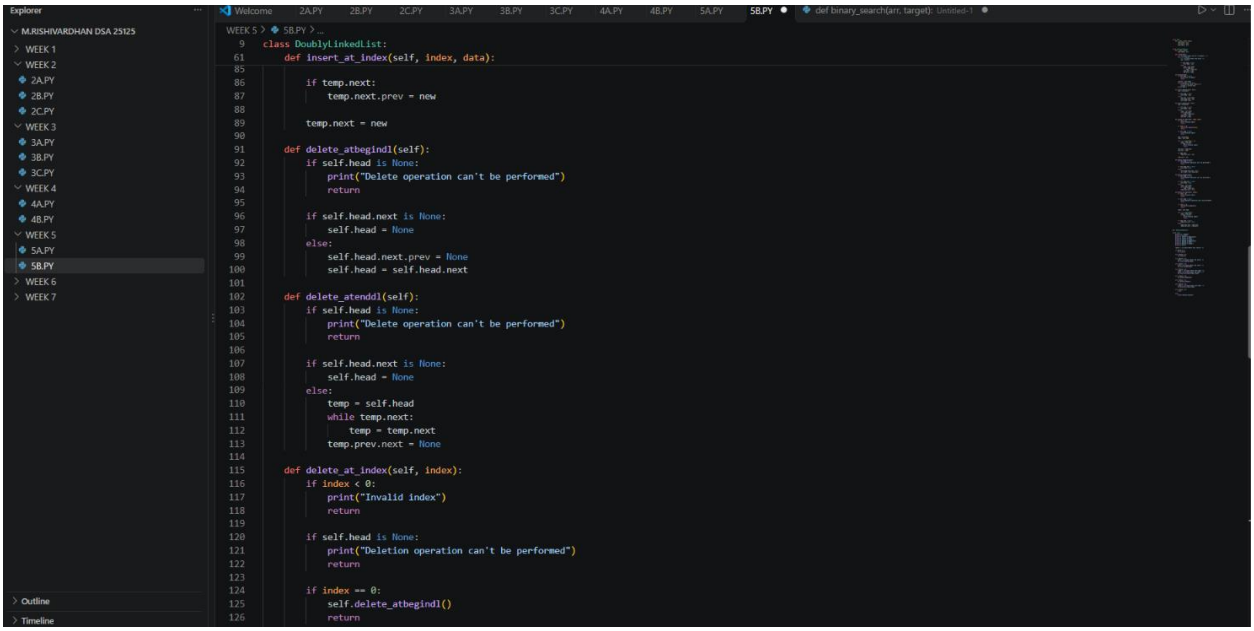
        if index == 0:
            self.insert_begindl(data)
            return

        if self.head is None:
            print("Invalid index")
            return

        new = Node(data)
        temp = self.head

        for i in range(index - 1):
            temp = temp.next
            if temp is None:
                print("Invalid index")
                return

        new.next = temp.next
        new.prev = temp
```



```
class DoublyLinkedList:
    def insert_at_index(self, index, data):

        if temp.next:
            temp.next.prev = new

        temp.next = new

    def delete_atbegindl(self):
        if self.head is None:
            print("Delete operation can't be performed")
            return

        if self.head.next is None:
            self.head = None
        else:
            self.head.next.prev = None
            self.head = self.head.next

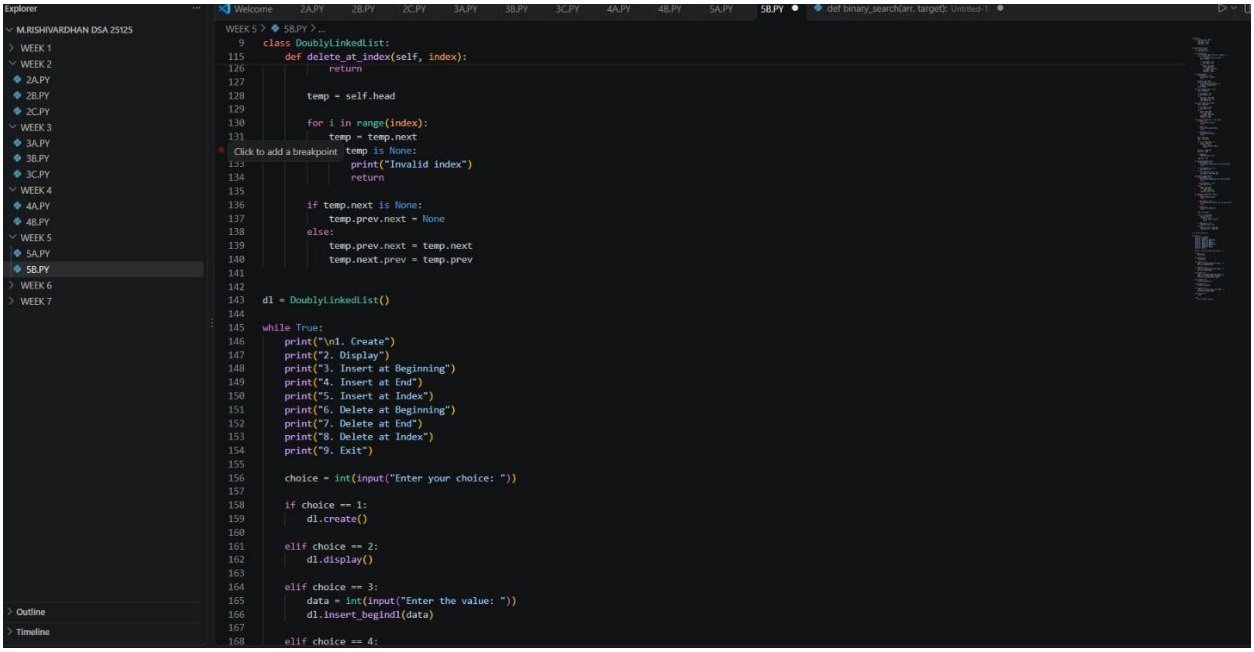
    def delete_atenddl(self):
        if self.head is None:
            print("Delete operation can't be performed")
            return

        if self.head.next is None:
            self.head = None
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.prev.next = None

    def delete_at_index(self, index):
        if index < 0:
            print("Invalid index")
            return

        if self.head is None:
            print("Deletion operation can't be performed")
            return

        if index == 0:
            self.delete_atbegindl()
            return
```



```
class DoublyLinkedList:
    def delete_at_index(self, index):
            return

        temp = self.head

        for i in range(index):
            temp = temp.next
            print("Invalid index")
            return

        if temp.next is None:
            temp.prev.next = None
        else:
            temp.prev.next = temp.next
            temp.next.prev = temp.prev


dl = DoublyLinkedList()

while True:
    print("\n1. Create")
    print("2. Display")
    print("3. Insert at Beginning")
    print("4. Insert at End")
    print("5. Insert at Index")
    print("6. Delete at Beginning")
    print("7. Delete at End")
    print("8. Delete at Index")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dl.create()

    elif choice == 2:
        dl.display()

    elif choice == 3:
        data = int(input("Enter the value: "))
        dl.insert_begindl(data)

    elif choice == 4:
```



```
        dl.display()

    elif choice == 3:
        data = int(input("Enter the value: "))
        dl.insert_begindl(data)

    elif choice == 4:
        data = int(input("Enter the value: "))
        dl.insert_enddl(data)

    elif choice == 5:
        index = int(input("Enter the index: "))
        data = int(input("Enter the value: "))
        dl.insert_at_index(index, data)

    elif choice == 6:
        dl.delete_atbegindl()

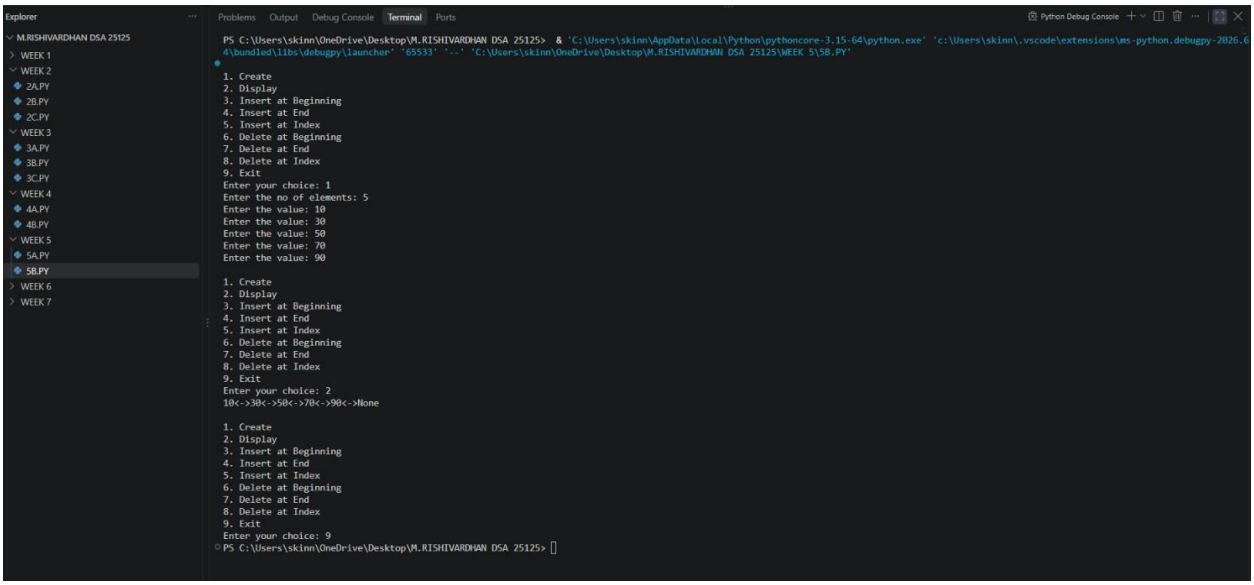
    elif choice == 7:
        dl.delete_atenddl()

    elif choice == 8:
        index = int(input("Enter the index: "))
        dl.delete_at_index(index)

    elif choice == 9:
        break

    else:
        print("Invalid choice")
```

Output:



```
1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 1
Enter the no of elements: 5
Enter the value: 10
Enter the value: 30
Enter the value: 50
Enter the value: 70
Enter the value: 90

1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 2
10<->30<->50<->70<->90<->None

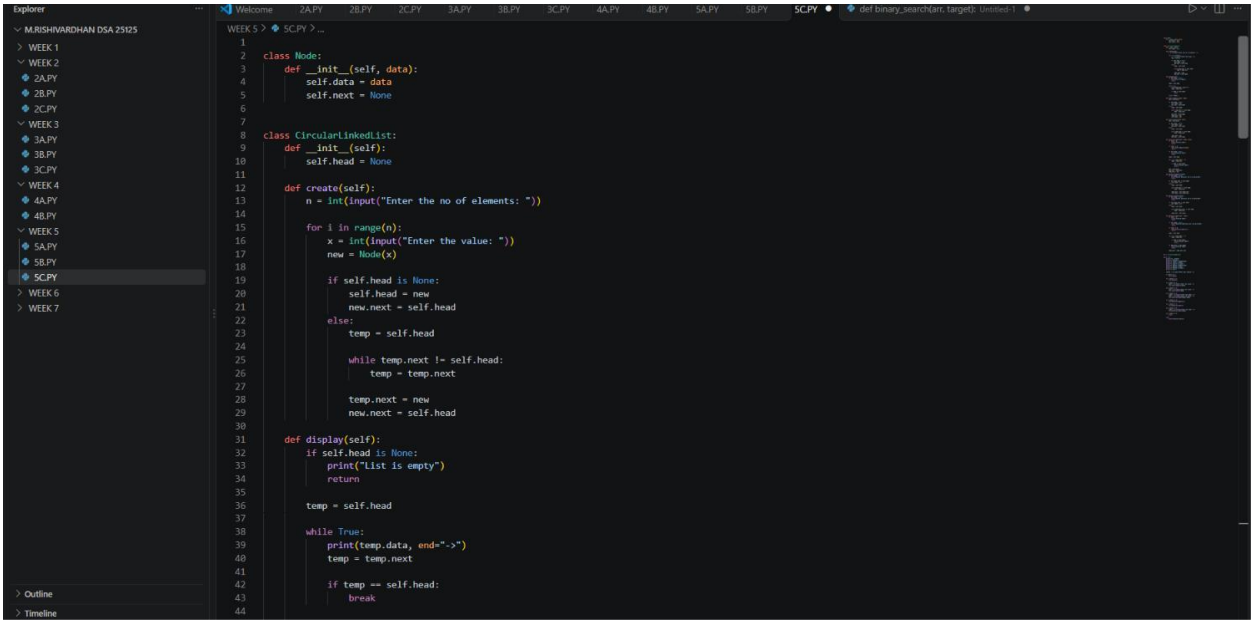
1. Create
2. Display
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Exit
Enter your choice: 9
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125>
```

Description:

This program implements a Doubly Linked List in which each node contains data along with pointers to both the next and the previous node. The menu-driven program supports create, display, insert at beginning/end/index, delete at beginning/end/index, and exit operations. The doubly linked structure allows traversal in both directions.

(c) Write a Python program to implement CLL with the basic operations

Code:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter the no of elements: "))

        for i in range(n):
            x = int(input("Enter the value: "))
            new = Node(x)

            if self.head is None:
                self.head = new
                new.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

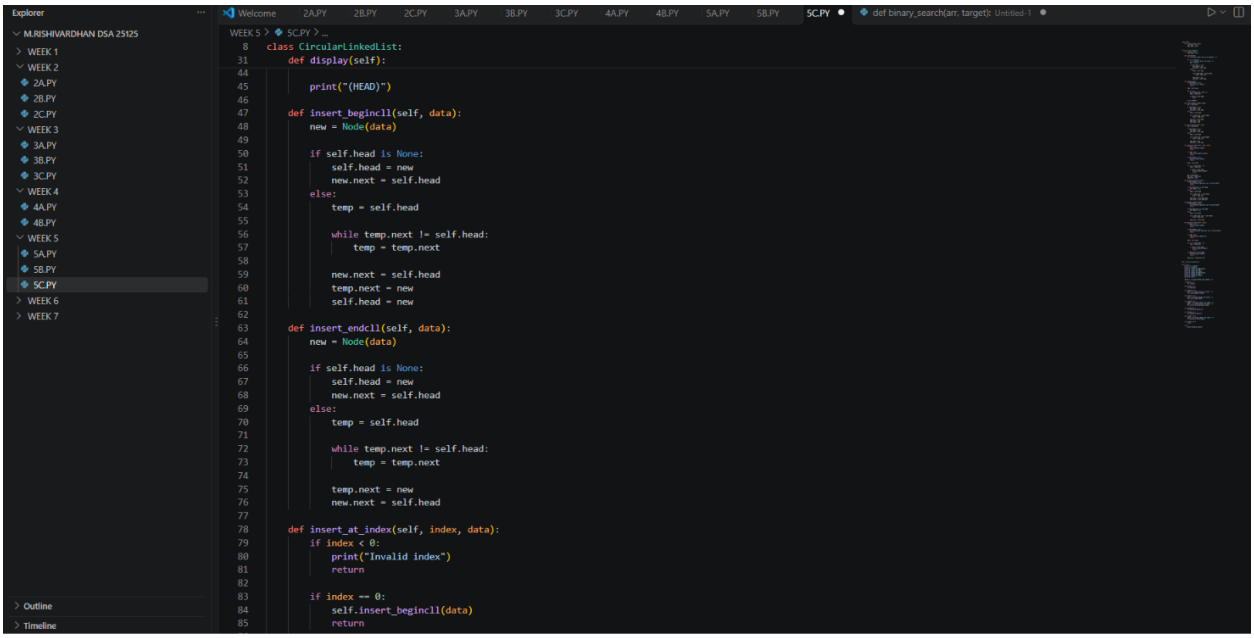
                temp.next = new
                new.next = self.head

    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while True:
            print(temp.data, end="->")
            temp = temp.next

            if temp == self.head:
                break
```



```
class CircularLinkedList:
    def display(self):

        print("(HEAD)")

    def insert_beginCll(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new.next = self.head
            temp.next = new
            self.head = new

    def insert_endCll(self, data):
        new = Node(data)

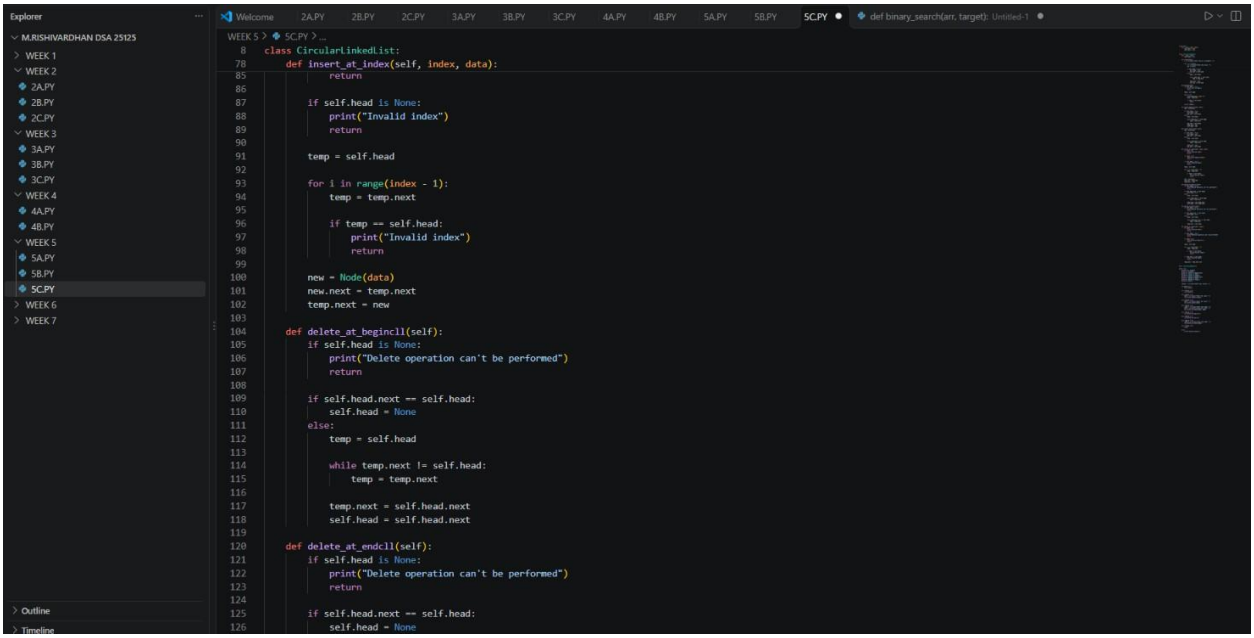
        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

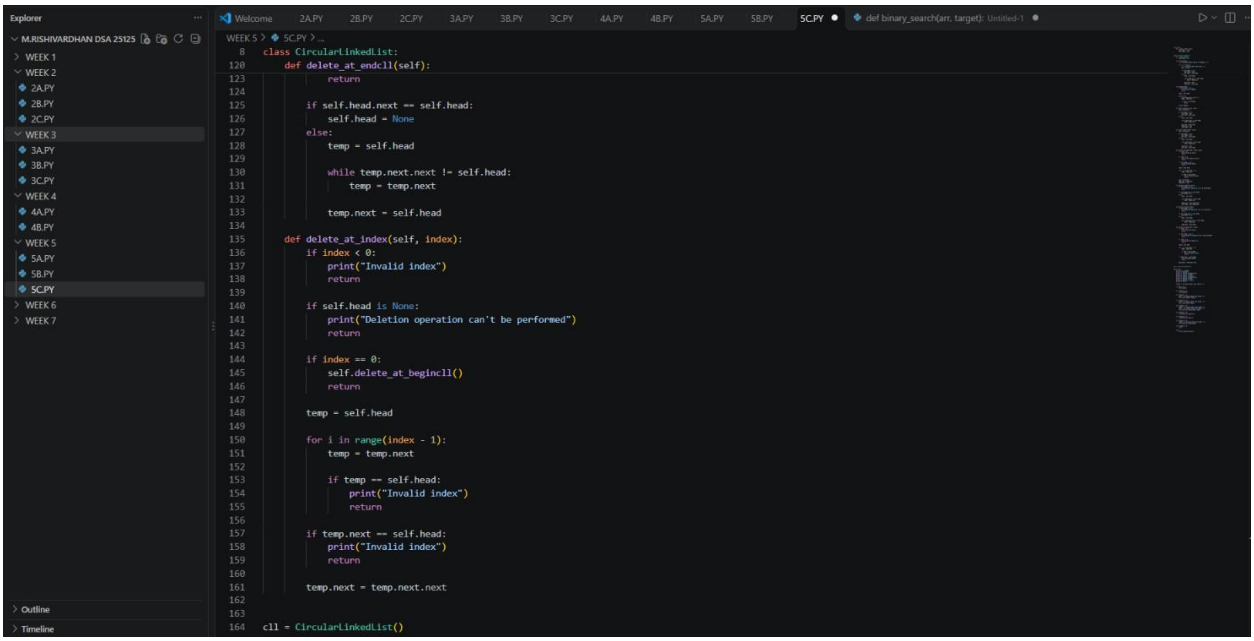
            while temp.next != self.head:
                temp = temp.next

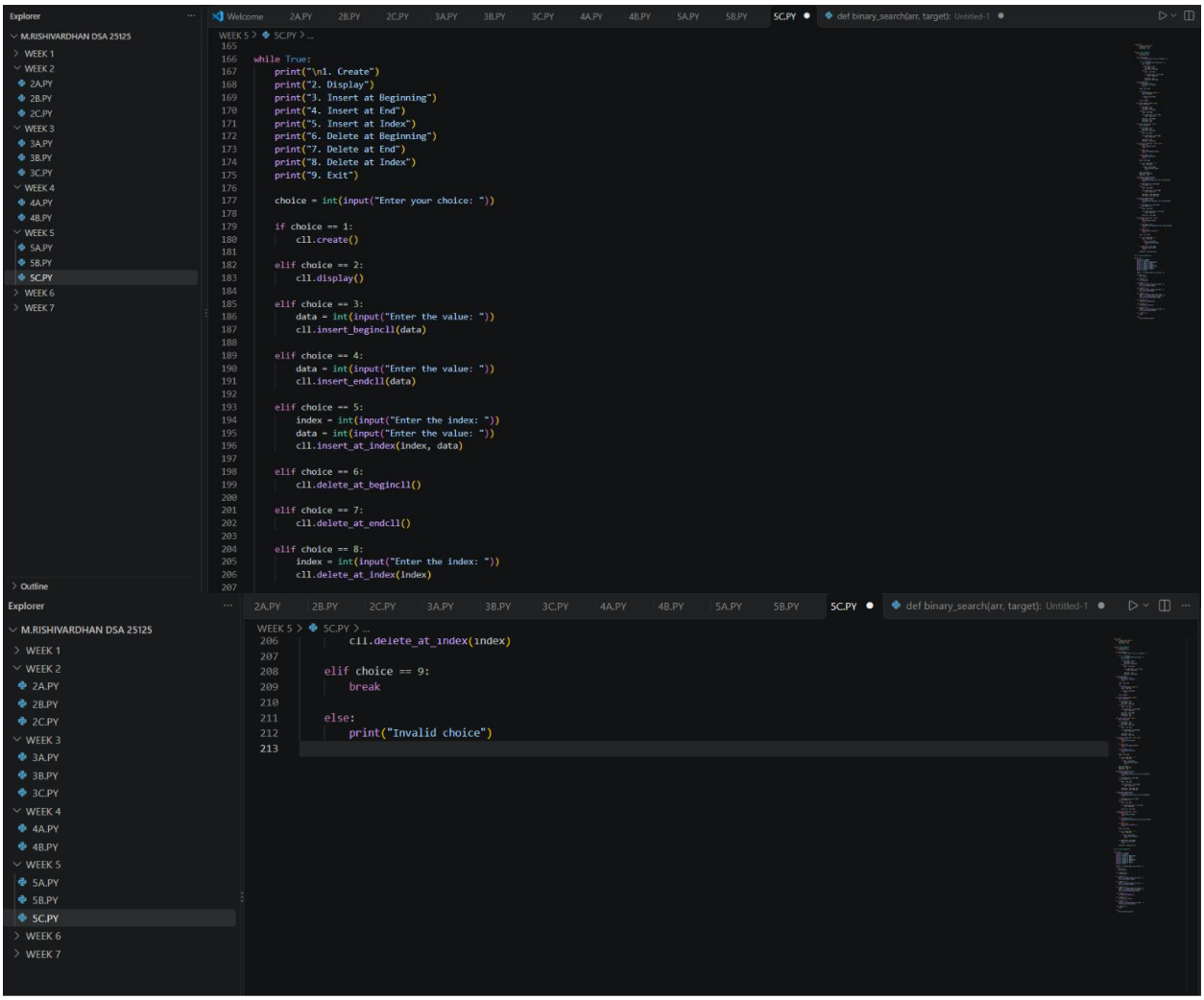
            temp.next = new
            new.next = self.head

    def insert_at_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return

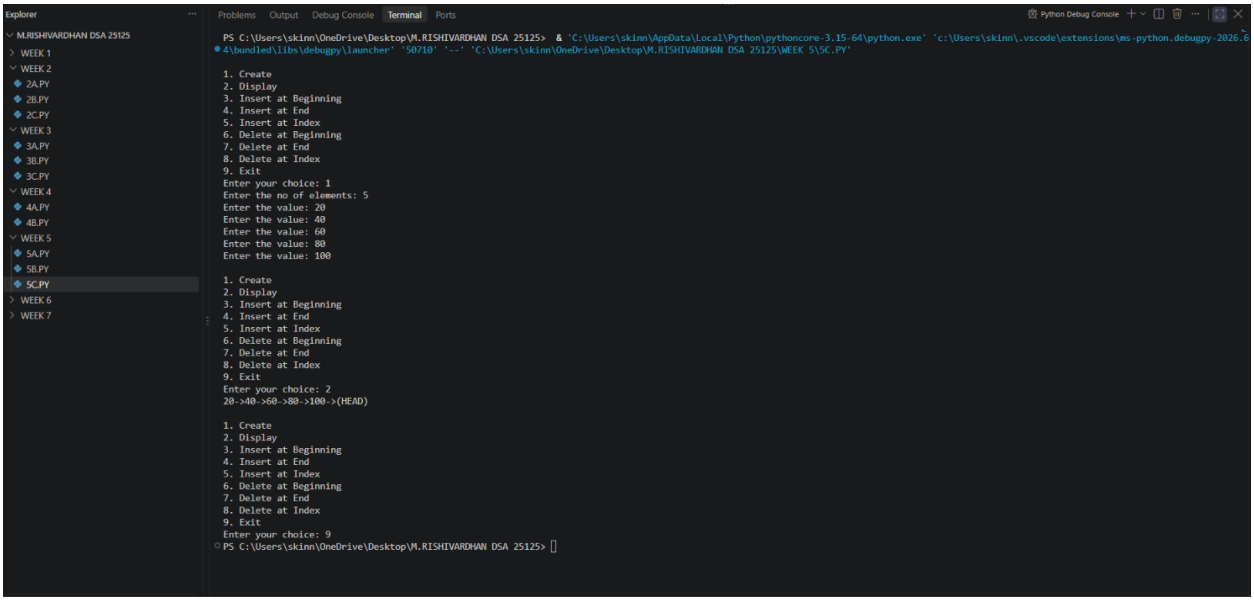
        if index == 0:
            self.insert_beginCll(data)
            return
```


Explorer
M.RISHIVARDHAN DSA 25125
WEEK 1
WEEK 2
2A.PY
2B.PY
2C.PY
WEEK 3
3A.PY
3B.PY
3C.PY
WEEK 4
4A.PY
4B.PY
WEEK 5
5A.PY
5B.PY
5C.PY
WEEK 6
WEEK 7
Outline
Timeline
Welcome
2A.PY
2B.PY
2C.PY
3A.PY
3B.PY
3C.PY
4A.PY
4B.PY
5A.PY
5B.PY
5C.PY
def binary_search(arr, target): Untitled-1
WEEK 5 > 5C.PY > ...
class CircularLinkedList:
def insert_at_index(self, index, data):
return
if self.head is None:
print("Invalid index")
return
temp = self.head
for i in range(index - 1):
temp = temp.next
if temp == self.head:
print("Invalid index")
return
new = Node(data)
new.next = temp.next
temp.next = new
def delete_at_begincll(self):
if self.head is None:
print("Delete operation can't be performed")
return
if self.head.next == self.head:
self.head = None
else:
temp = self.head
while temp.next != self.head:
temp = temp.next
temp.next = self.head.next
self.head = self.head.next
def delete_at_endcll(self):
if self.head is None:
print("Delete operation can't be performed")
return
if self.head.next == self.head:
self.head = None


Explorer
M.RISHIVARDHAN DSA 25125
WEEK 1
WEEK 2
2A.PY
2B.PY
2C.PY
WEEK 3
3A.PY
3B.PY
3C.PY
WEEK 4
4A.PY
4B.PY
WEEK 5
5A.PY
5B.PY
5C.PY
WEEK 6
WEEK 7
Outline
Timeline
Welcome
2A.PY
2B.PY
2C.PY
3A.PY
3B.PY
3C.PY
4A.PY
4B.PY
5A.PY
5B.PY
5C.PY
def binary_search(arr, target): Untitled-1
WEEK 5 > 5C.PY > ...
class CircularLinkedList:
def delete_at_endcll(self):
return
if self.head.next == self.head:
self.head = None
else:
temp = self.head
while temp.next.next != self.head:
temp = temp.next
temp.next = self.head
def delete_at_index(self, index):
if index < 0:
print("Invalid index")
return
if self.head is None:
print("Deletion operation can't be performed")
return
if index == 0:
self.delete_at_begincll()
return
temp = self.head
for i in range(index - 1):
temp = temp.next
if temp == self.head:
print("Invalid index")
return
if temp.next == self.head:
print("Invalid index")
return
temp.next = temp.next.next
cll = CircularLinkedList()


Explorer
M.RISHIVARDHAN DSA 25125
WEEK 1
WEEK 2
2A.PY
2B.PY
2C.PY
WEEK 3
3A.PY
3B.PY
3C.PY
WEEK 4
4A.PY
4B.PY
WEEK 5
5A.PY
5B.PY
5C.PY
WEEK 6
WEEK 7
Outline
Welcome
2A.PY
2B.PY
2C.PY
3A.PY
3B.PY
3C.PY
4A.PY
4B.PY
5A.PY
5B.PY
5C.PY
def binary_search(arr, target): Untitled-1
WEEK 5 > 5C.PY > ...
while True:
print("\n1. Create")
print("2. Display")
print("3. Insert at Beginning")
print("4. Insert at End")
print("5. Insert at Index")
print("6. Delete at Beginning")
print("7. Delete at End")
print("8. Delete at Index")
print("9. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
cll.create()
elif choice == 2:
cll.display()
elif choice == 3:
data = int(input("Enter the value: "))
cll.insert_begincll(data)
elif choice == 4:
data = int(input("Enter the value: "))
cll.insert_endcll(data)
elif choice == 5:
index = int(input("Enter the index: "))
data = int(input("Enter the value: "))
cll.insert_at_index(index, data)
elif choice == 6:
cll.delete_at_begincll()
elif choice == 7:
cll.delete_at_endcll()
elif choice == 8:
index = int(input("Enter the index: "))
cll.delete_at_index(index)
cll.delete_at_index(index)
elif choice == 9:
break
else:
print("Invalid choice")

Output:



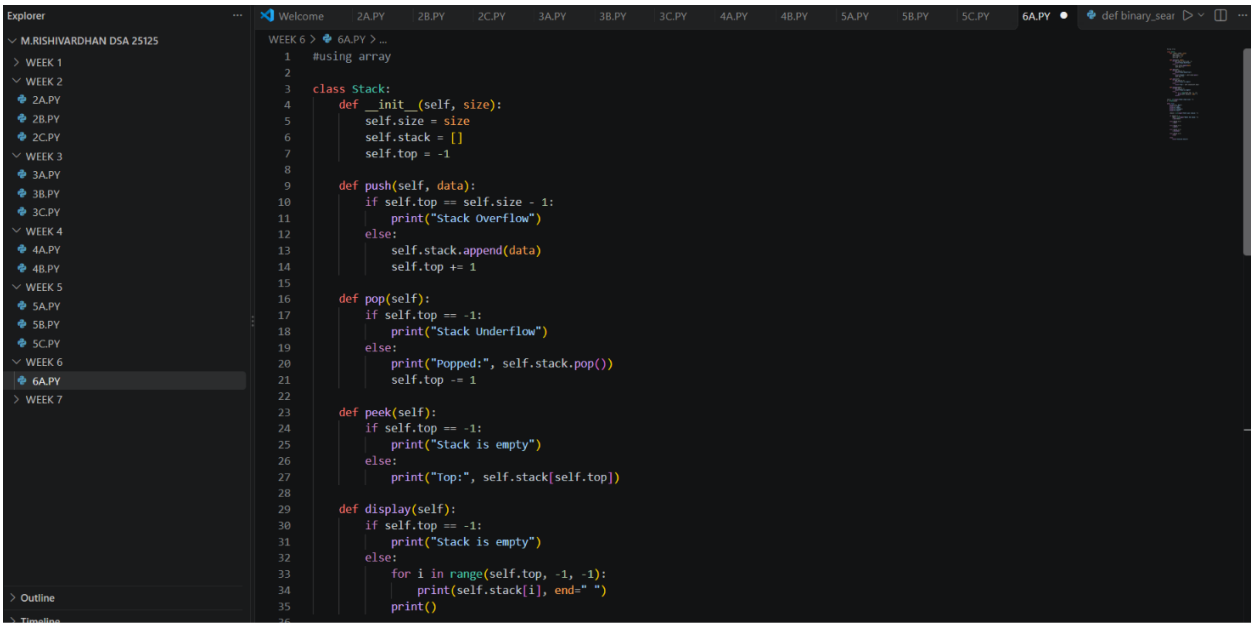
Description:

This program implements a Circular Linked List in which the last node points back to the head node, forming a circle. The menu-driven program supports create, display, insert at beginning/end/index, delete at beginning/end/index, and exit operations. Traversal stops when the pointer returns to the head.

Week-6

(a) Write a Python program to implement Stack operations such as Push, Pop, Peek, and Display using an array.

Code:



```
#using array

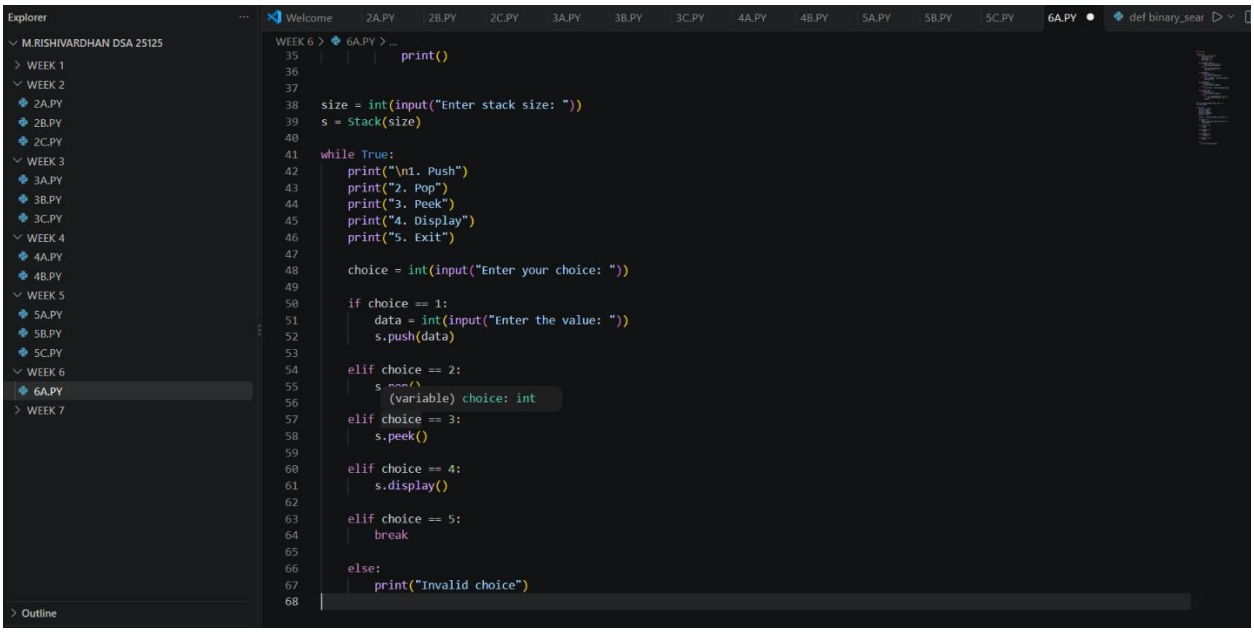
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = []
        self.top = -1

    def push(self, data):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.stack.append(data)
            self.top += 1

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            print("Popped:", self.stack.pop())
            self.top -= 1

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            for i in range(self.top, -1, -1):
                print(self.stack[i], end=" ")
            print()
```



```
            print()


size = int(input("Enter stack size: "))
s = Stack(size)

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the value: "))
        s.push(data)

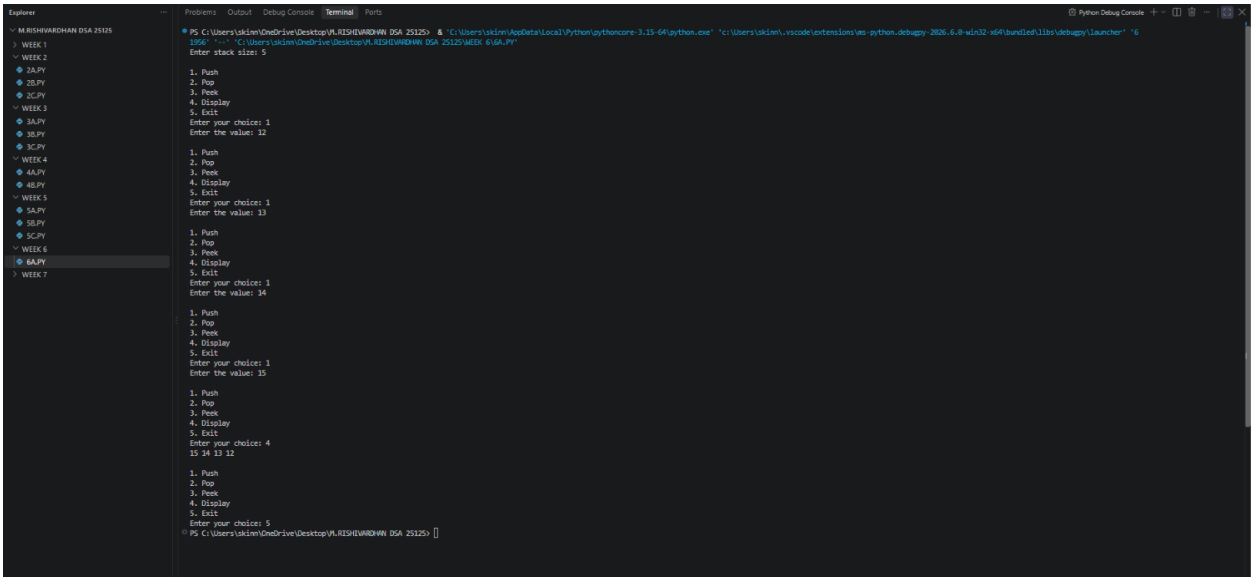
    elif choice == 2:
        s.pop()
            (variable) choice: int
    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

Output:

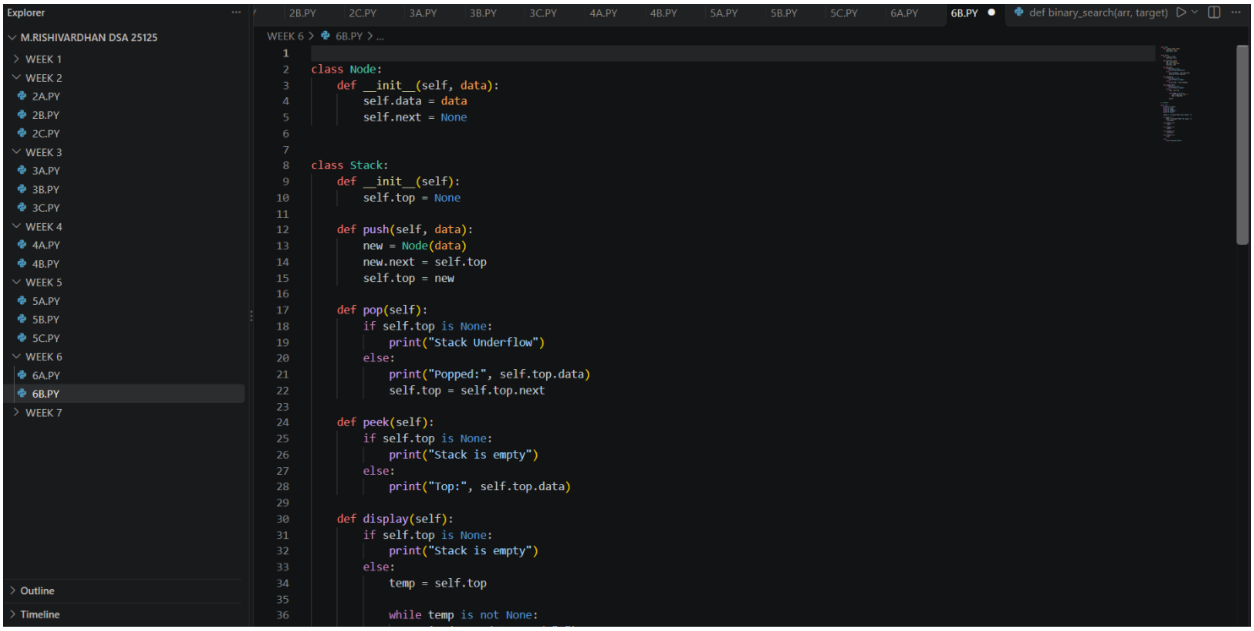


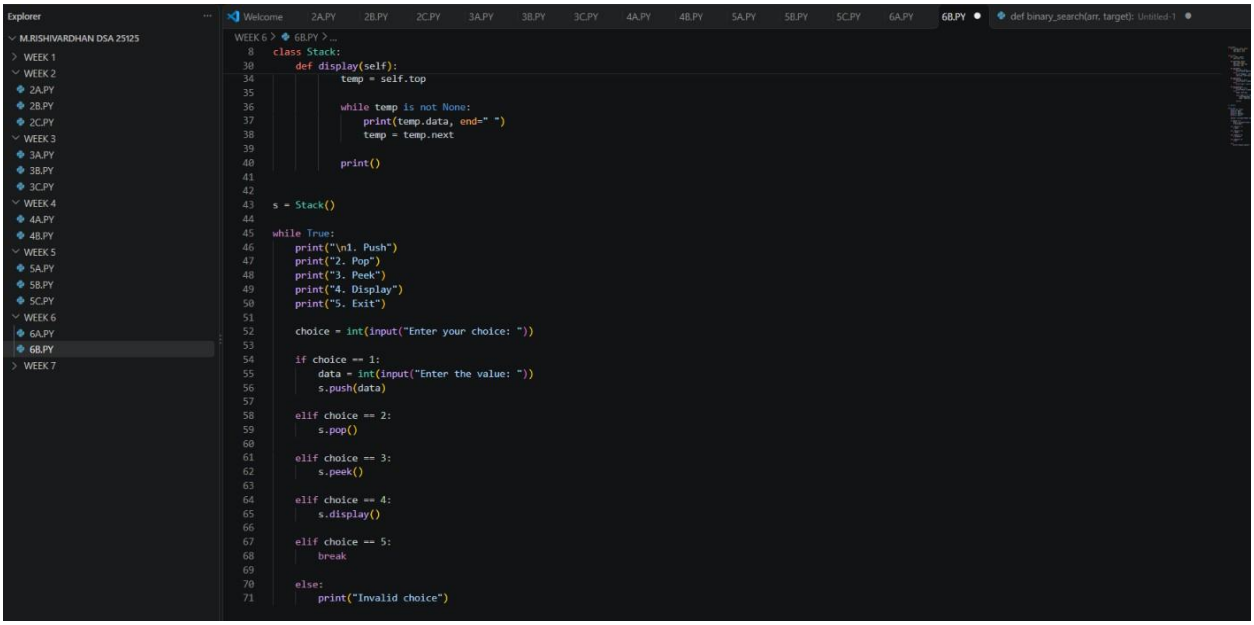
Description:

This program implements a Stack using an array following the LIFO (Last In First Out) principle. The push operation inserts an element at the top of the stack, the pop operation removes the top element, peek returns the top element without removing it, and display shows all elements of the stack from top to bottom. The program also handles stack overflow and underflow conditions.

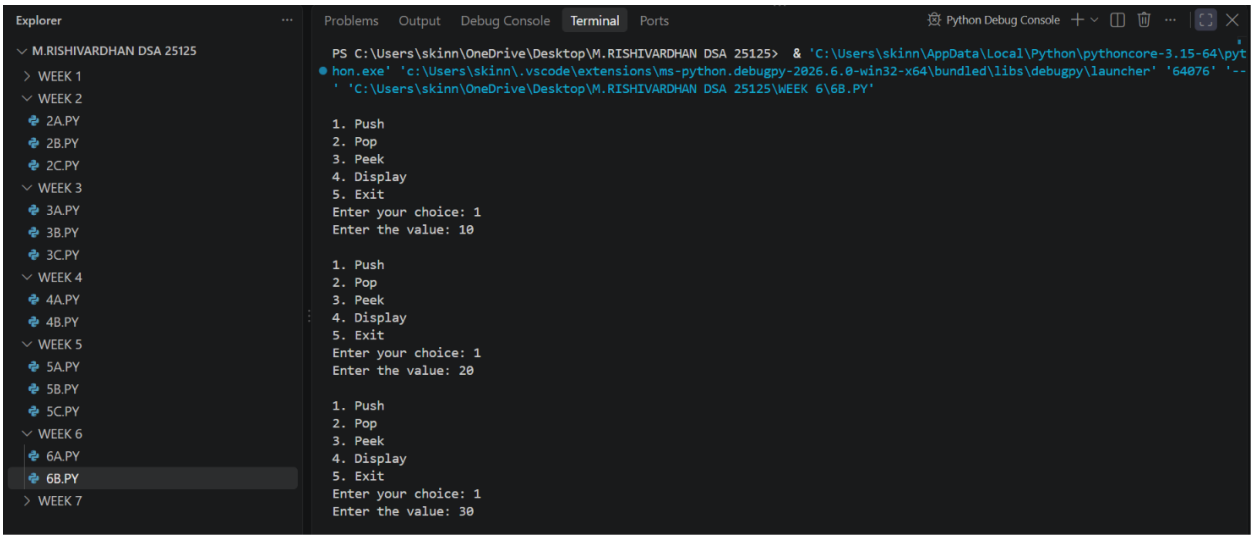
(b) Write a Python program to implement Stack operations such as Push, Pop, Peek using linked list.

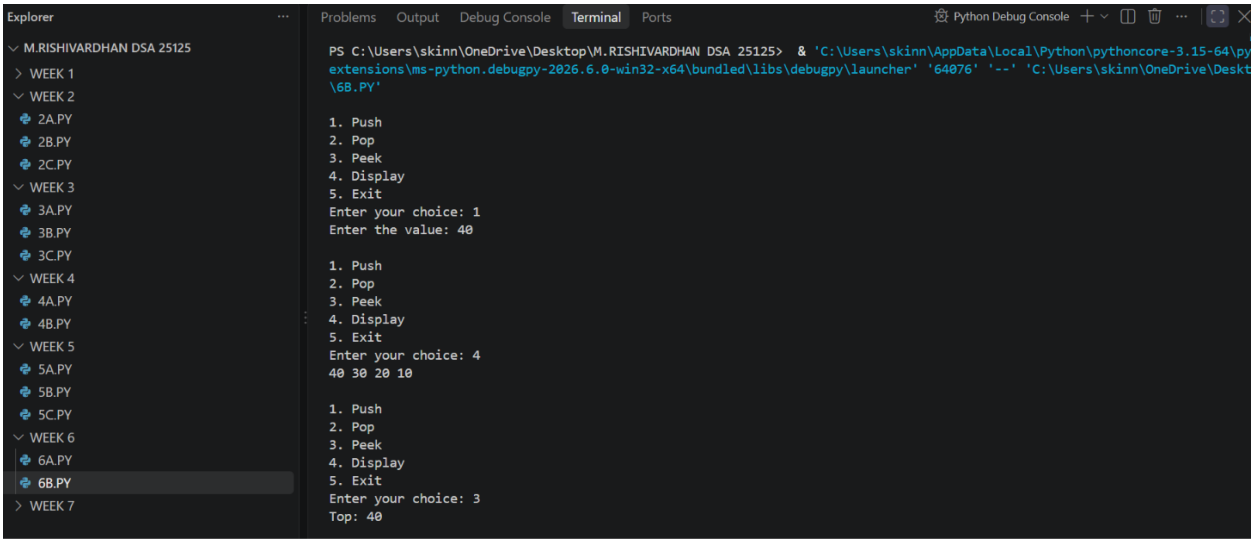
Code:

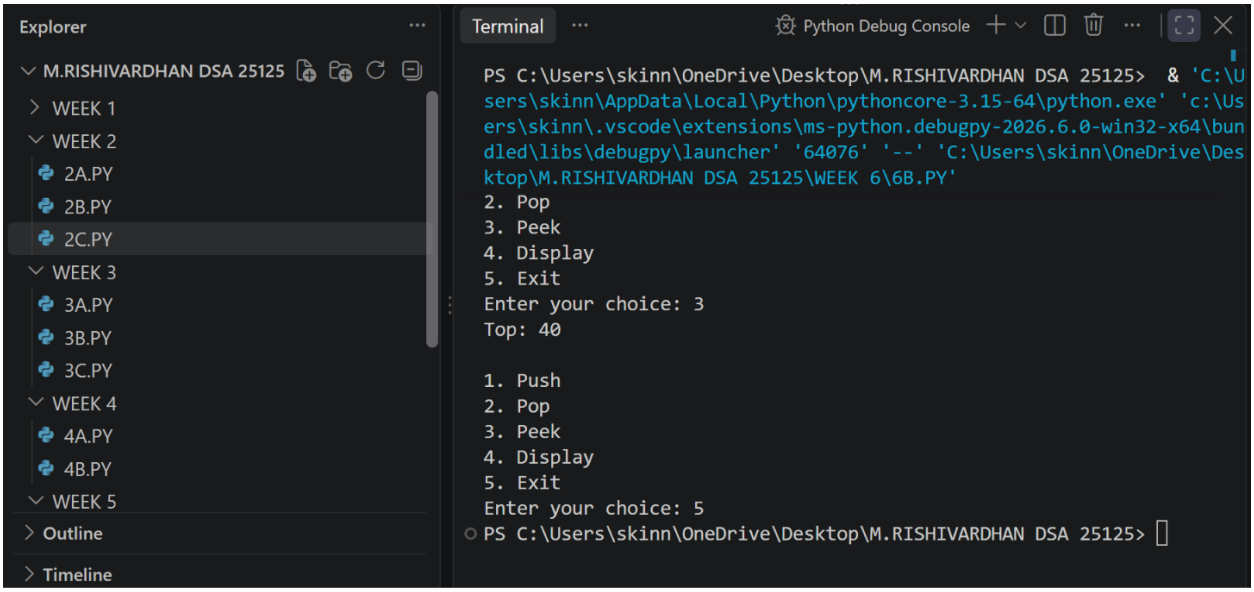



Explorer
M.RISHIVARDHAN DSA 25125
WEEK 1
WEEK 2
2A.PY
2B.PY
2C.PY
WEEK 3
3A.PY
3B.PY
3C.PY
WEEK 4
4A.PY
4B.PY
WEEK 5
5A.PY
5B.PY
5C.PY
WEEK 6
6A.PY
6B.PY
WEEK 7
Welcome
2A.PY
2B.PY
2C.PY
3A.PY
3B.PY
3C.PY
4A.PY
4B.PY
5A.PY
5B.PY
5C.PY
6A.PY
6B.PY
def binary_search(arr, target): Untitled-1
WEEK 6 > 6B.PY > ...
class Stack:
def display(self):
temp = self.top
while temp is not None:
print(temp.data, end=" ")
temp = temp.next
print()
s = Stack()
while True:
print("\n1. Push")
print("2. Pop")
print("3. Peek")
print("4. Display")
print("5. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
data = int(input("Enter the value: "))
s.push(data)
elif choice == 2:
s.pop()
elif choice == 3:
s.peek()
elif choice == 4:
s.display()
elif choice == 5:
break
else:
print("Invalid choice")

Output:


Explorer
M.RISHIVARDHAN DSA 25125
WEEK 1
WEEK 2
2A.PY
2B.PY
2C.PY
WEEK 3
3A.PY
3B.PY
3C.PY
WEEK 4
4A.PY
4B.PY
WEEK 5
5A.PY
5B.PY
5C.PY
WEEK 6
6A.PY
6B.PY
WEEK 7
Problems
Output
Debug Console
Terminal
Ports
Python Debug Console
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125> & 'C:\Users\skinn\AppData\Local\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '64076' '--' 'C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 6\6B.PY'
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 10
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 20
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the value: 30





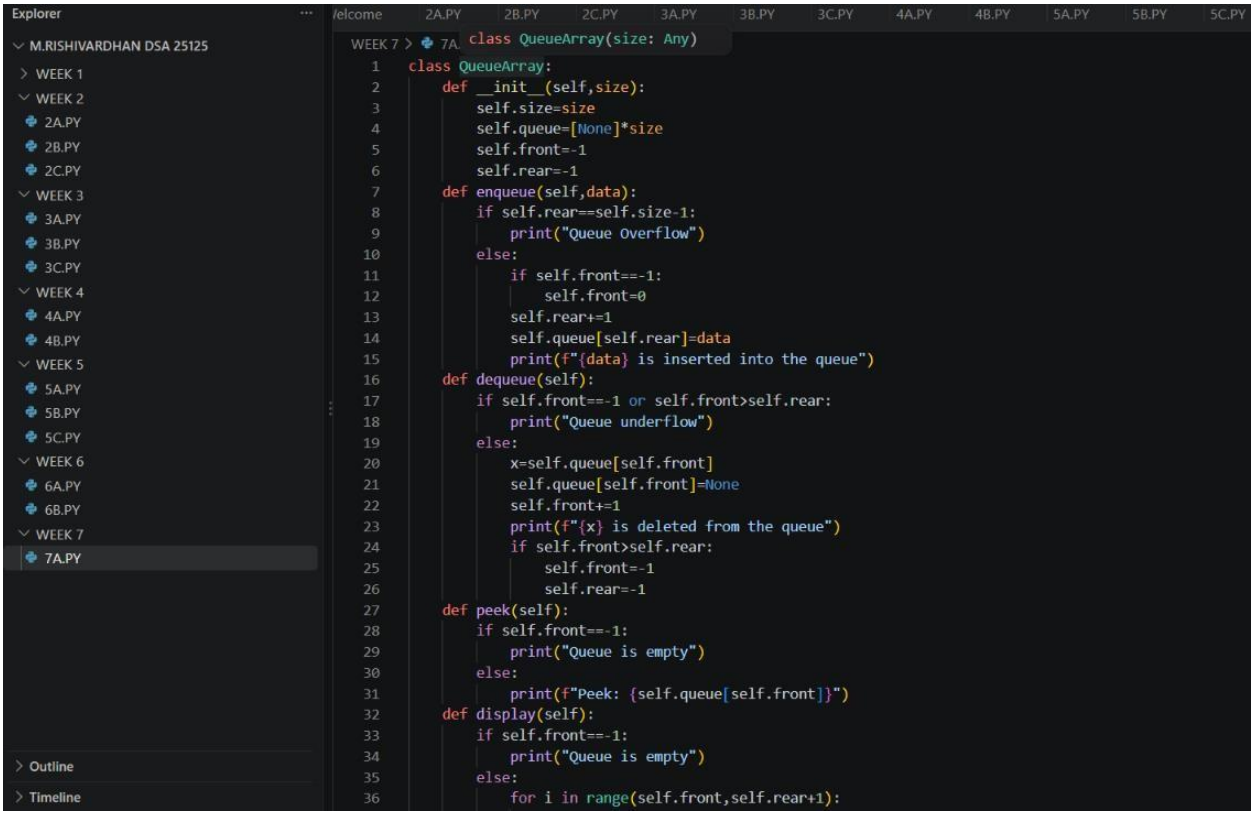
Description:

This program implements a Stack using a linked list following the LIFO (Last In First Out) principle. The top pointer always refers to the head node of the list. Push inserts a new node at the top, pop removes the top node, peek displays the top element, and display prints all elements from top to bottom. Unlike the array implementation, this version grows dynamically and does not have a fixed size.

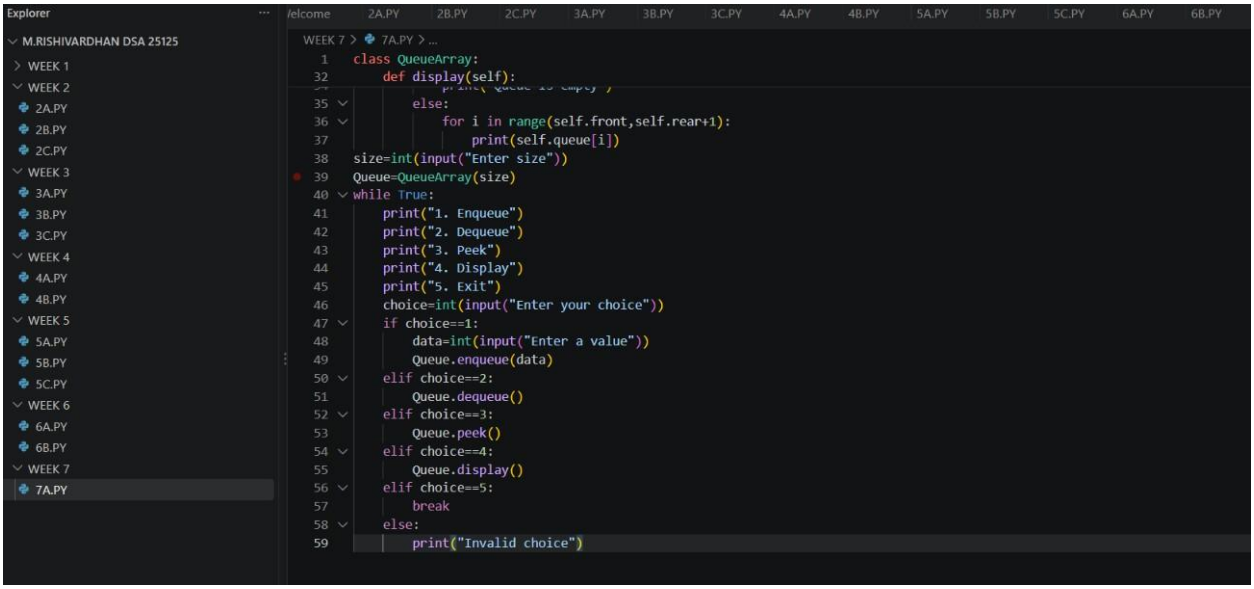
Week-7

(a) Write a Python program to implement Queue using Array.

Code:

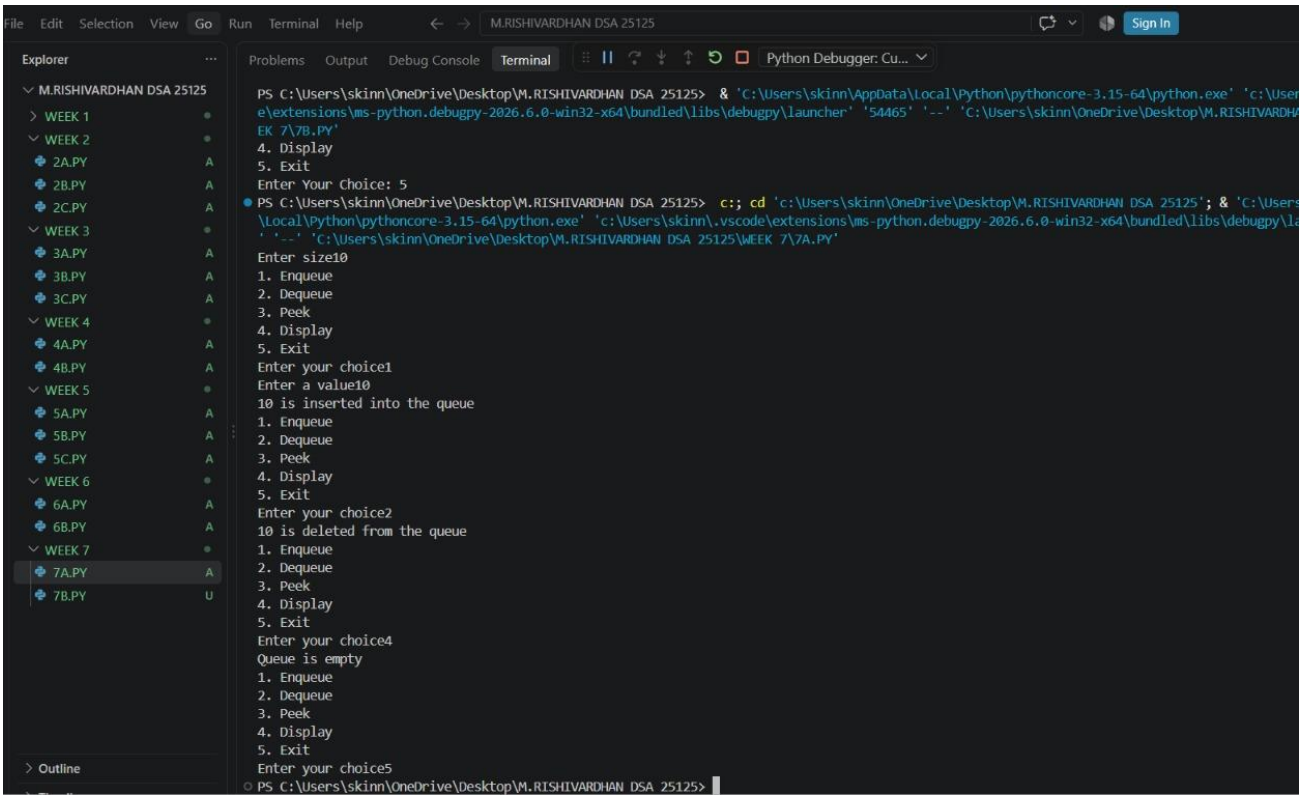


```
class QueueArray:
    def __init__(self,size):
        self.size=size
        self.queue=[None]*size
        self.front=-1
        self.rear=-1
    def enqueue(self,data):
        if self.rear==self.size-1:
            print("Queue Overflow")
        else:
            if self.front==-1:
                self.front=0
            self.rear+=1
            self.queue[self.rear]=data
            print(f"{data} is inserted into the queue")
    def dequeue(self):
        if self.front==-1 or self.front>self.rear:
            print("Queue underflow")
        else:
            x=self.queue[self.front]
            self.queue[self.front]=None
            self.front+=1
            print(f"{x} is deleted from the queue")
            if self.front>self.rear:
                self.front=-1
                self.rear=-1
    def peek(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            print(f"Peek: {self.queue[self.front]}")
    def display(self):
        if self.front==-1:
            print("Queue is empty")
        else:
            for i in range(self.front,self.rear+1):
```



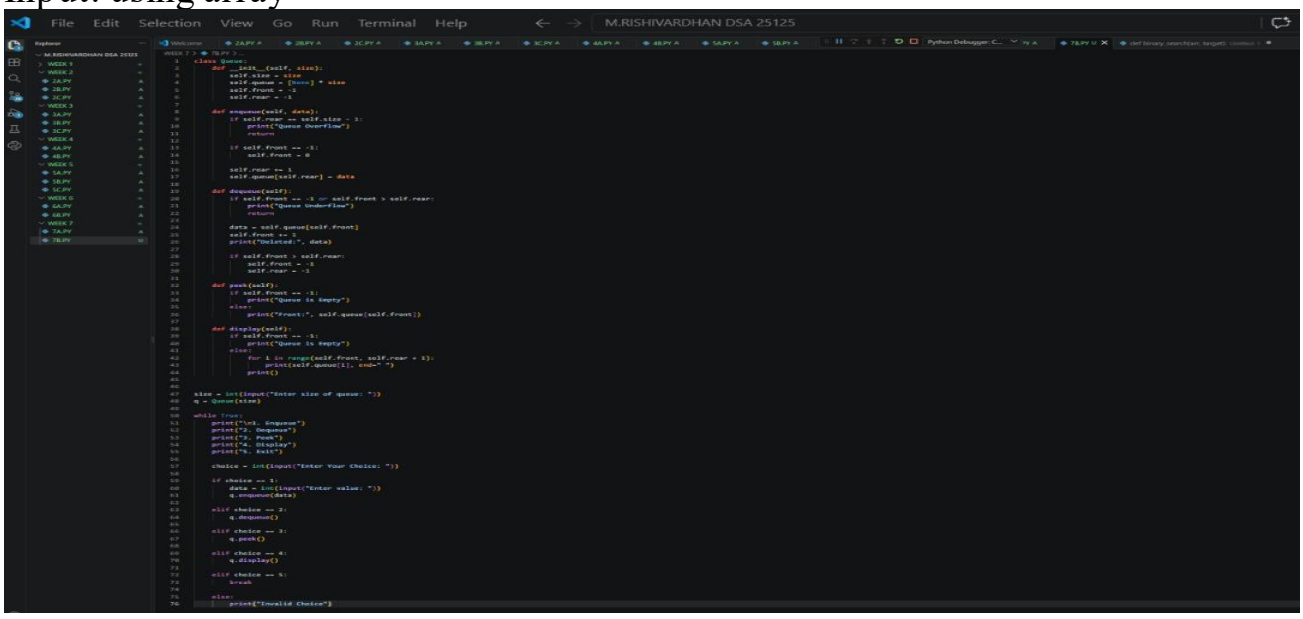
```
class QueueArray:
    def display(self):
        else:
            for i in range(self.front,self.rear+1):
                print(self.queue[i])
size=int(input("Enter size"))
Queue=QueueArray(size)
while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice=int(input("Enter your choice"))
    if choice==1:
        data=int(input("Enter a value"))
        Queue.enqueue(data)
    elif choice==2:
        Queue.dequeue()
    elif choice==3:
        Queue.peek()
    elif choice==4:
        Queue.display()
    elif choice==5:
        break
    else:
        print("Invalid choice")
```

Output:

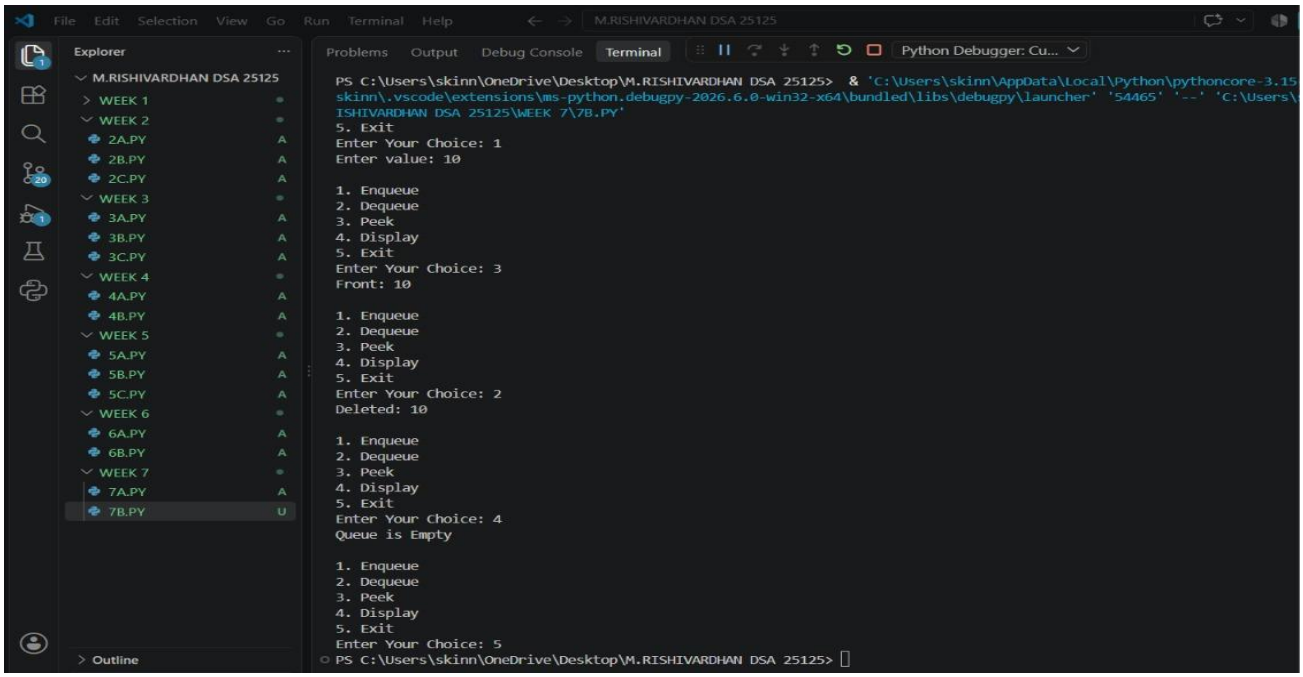


(b) Implement using array and SLL.

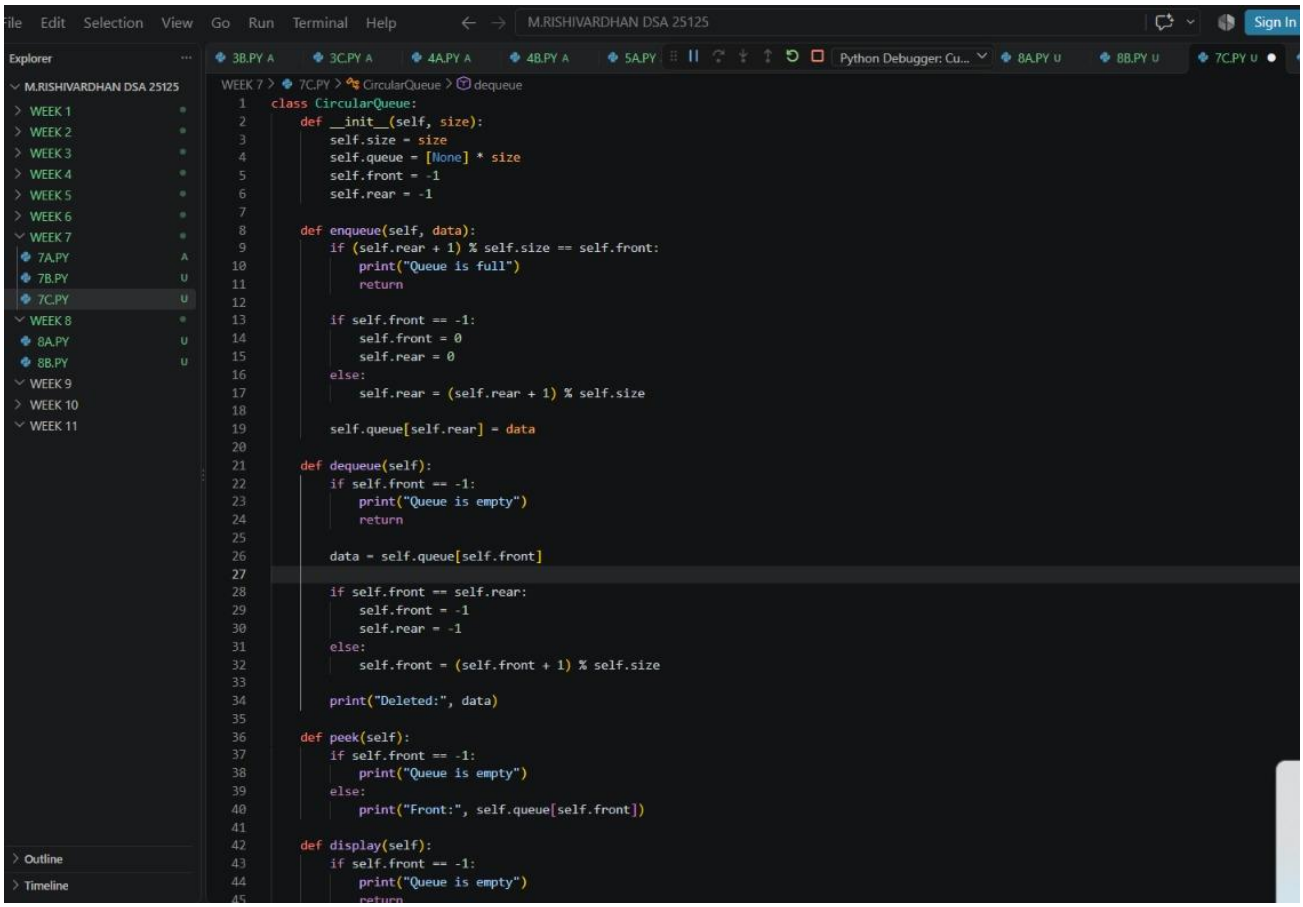
Input: using array



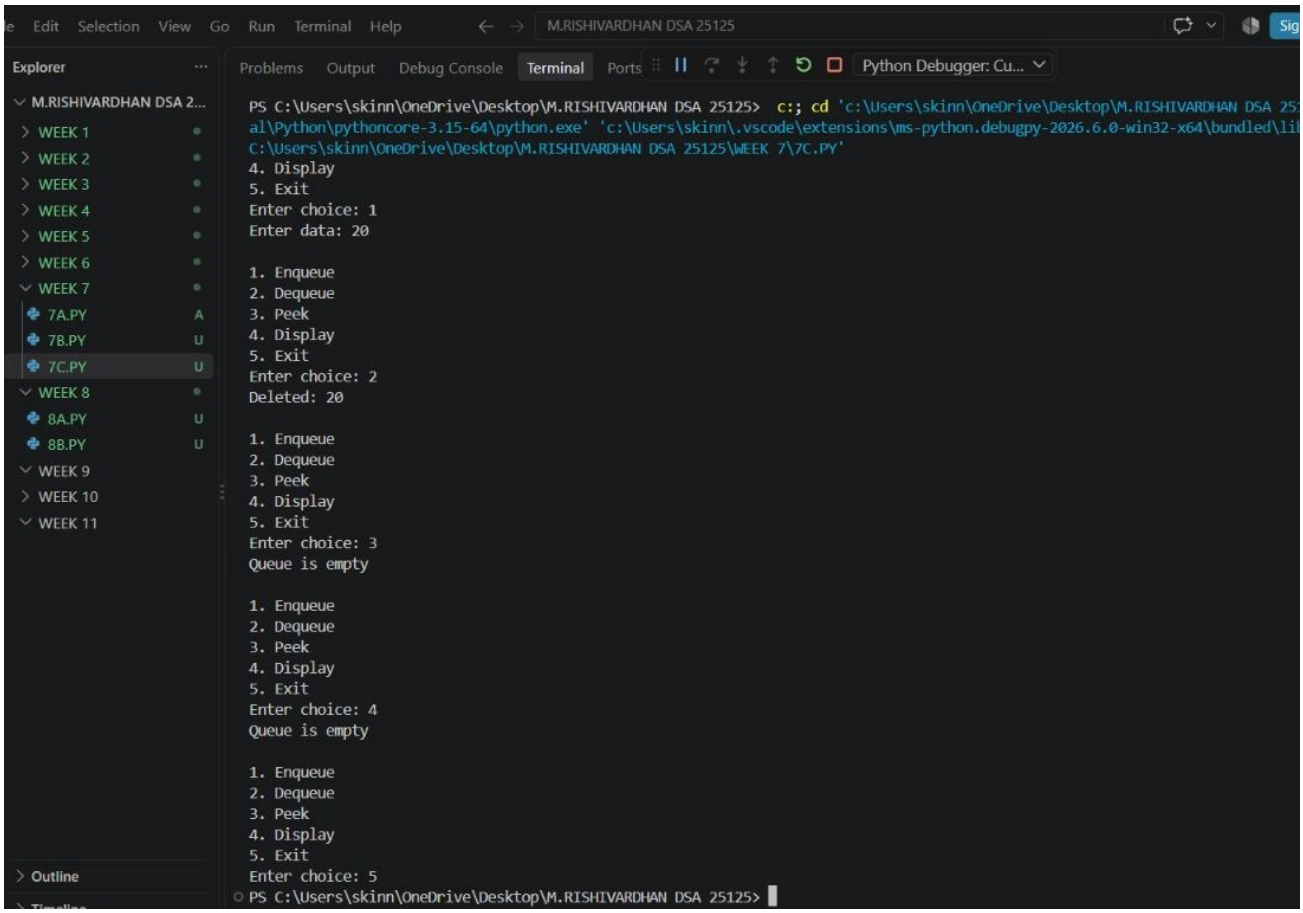
Output:



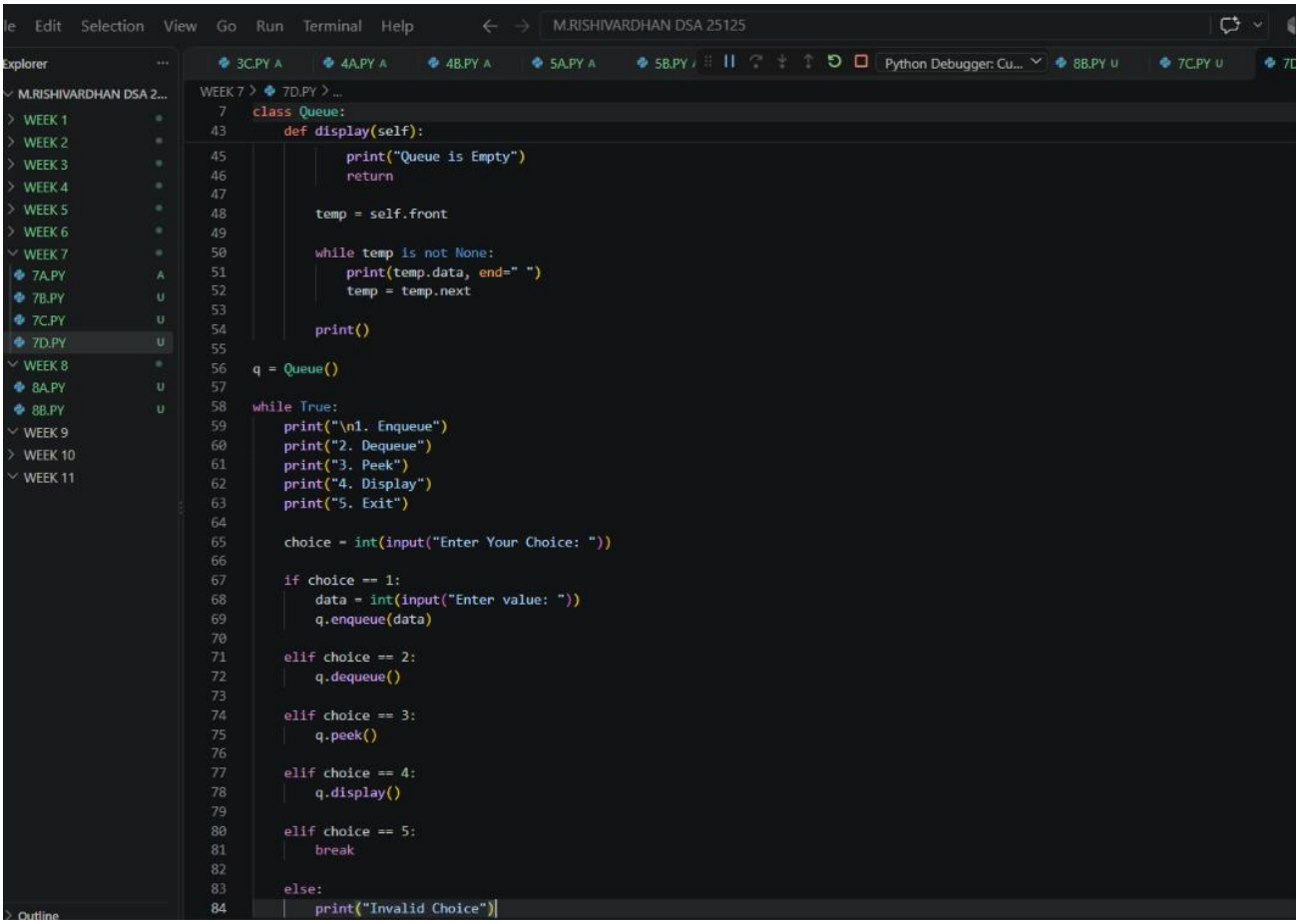
(c) Write a Python Program to implement Circular Queue



Output :


Edit Selection View Go Run Terminal Help
M.RISHIVARDHAN DSA 25125
Explorer
M.RISHIVARDHAN DSA 2...
WEEK 1
WEEK 2
WEEK 3
WEEK 4
WEEK 5
WEEK 6
WEEK 7
7A.PY
7B.PY
7C.PY
WEEK 8
8A.PY
8B.PY
WEEK 9
WEEK 10
WEEK 11
Outline
Problems Output Debug Console Terminal Ports
Python Debugger: Cu...
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125> c:; cd 'c:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25
al\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\li
C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 7\7C.PY'
4. Display
5. Exit
Enter choice: 1
Enter data: 20
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 2
Deleted: 20
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 3
Queue is empty
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 4
Queue is empty
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 5
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125>

Using SLL:



Description:

A queue is a linear data structure that follows the FIFO (First In, First Out) principle, in which the element inserted first is removed first. This program implements a queue using an array and a singly linked list (SLL) with a menu-driven approach.

The program supports the following operations:

1. **Enqueue:** Inserts a new element at the rear end of the queue.
2. **Dequeue:** Removes an element from the front of the queue. If the queue is empty, an appropriate message is displayed.
3. **Peek:** Displays the front element of the queue without removing it.
4. **Display:** Displays all the elements present in the queue.

In the array implementation, elements are stored in an array, and front and rear indices are used to manage the queue operations. In the singly linked list implementation, each node contains data and a pointer to the next node. The front and rear pointers are used to insert and delete elements efficiently.

The menu-driven approach allows the user to select an operation repeatedly until the exit option is

chosen.

A queue is a linear data structure that follows the FIFO (First In, First Out) principle, in which the element inserted first is removed first. A circular queue is an extension of a queue in which the last position is connected to the first position, allowing unused spaces in the array to be reused efficiently.

This program implements queue operations using an array, a circular array, and a singly linked list (SLL) with a menu-driven approach. The following operations are performed:

1. **Enqueue:** Inserts a new element at the rear end of the queue.
2. **Dequeue:** Removes an element from the front of the queue.
3. **Peek:** Displays the front element without removing it.
4. **Display:** Displays all the elements present in the queue.

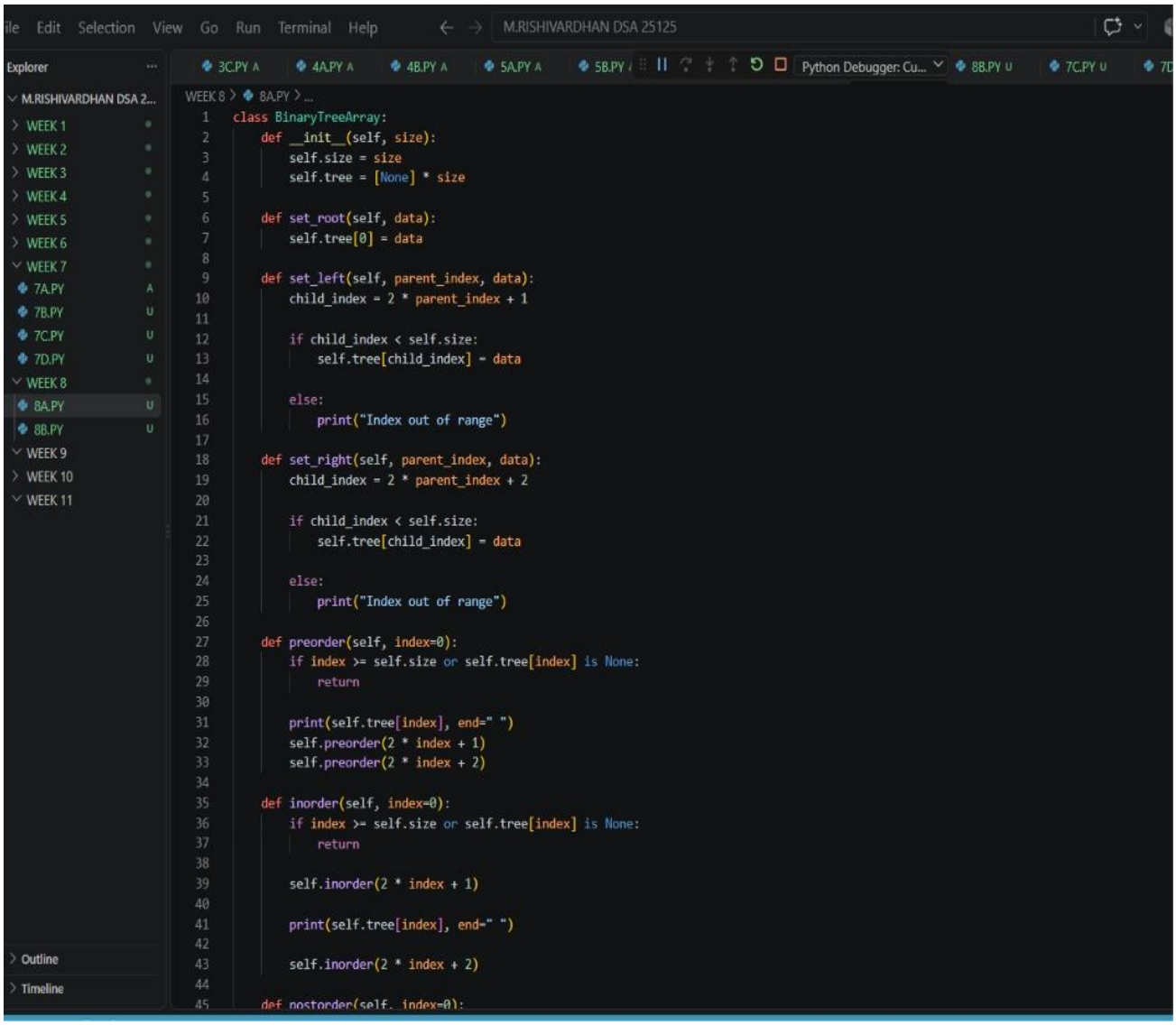
In the array implementation, front and rear indices are used to manage the elements. In the circular queue implementation, the indices wrap around to the beginning of the array when the end is reached, allowing available spaces to be reused. In the singly linked list implementation, each node contains data and a pointer to the next node, while front and rear pointers manage deletion and insertion.

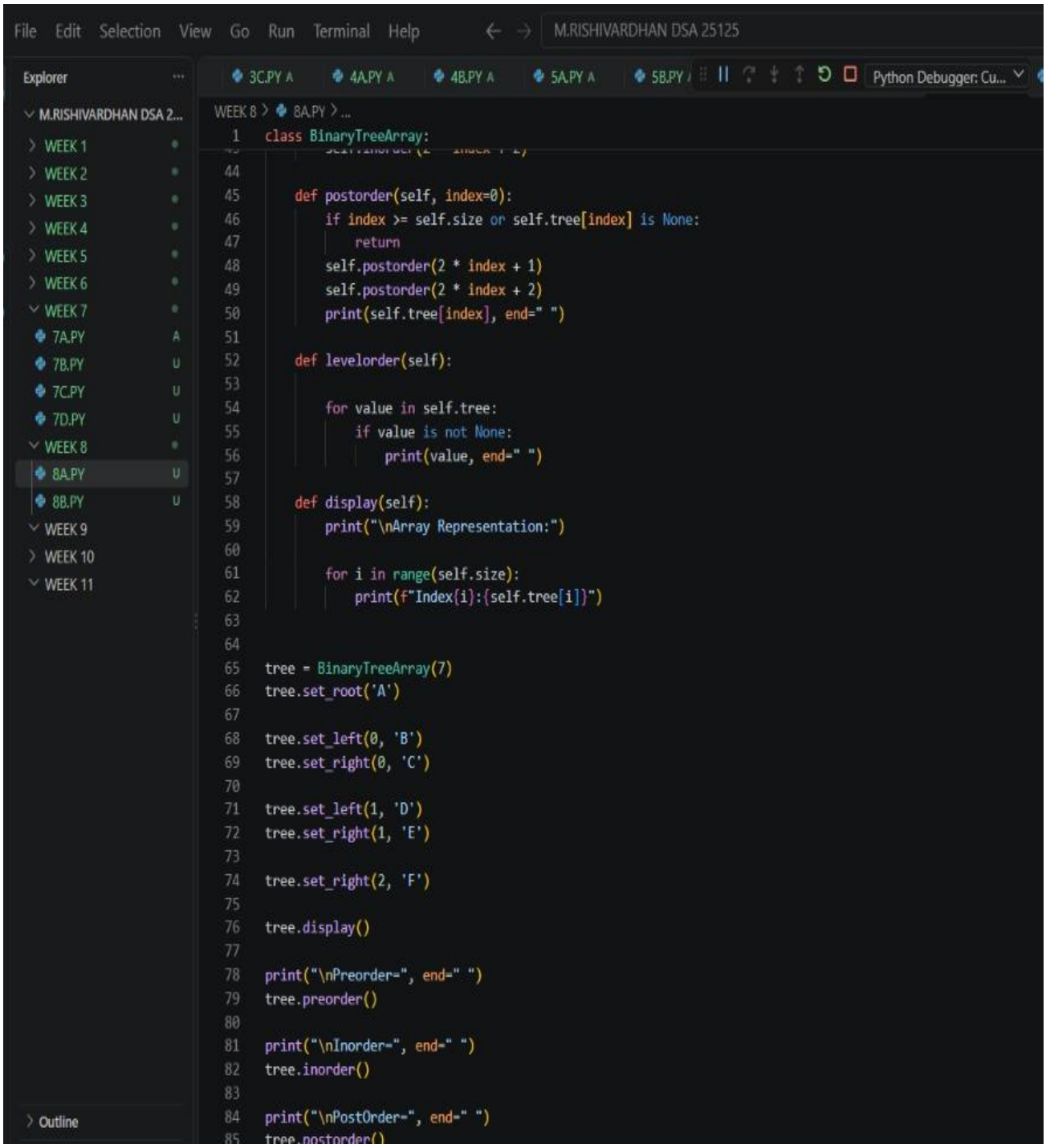
The program checks for overflow and underflow conditions wherever applicable and displays appropriate messages when the queue is full or empty. The menu-driven approach allows the user to perform operations repeatedly until the exit option is selected.

WEEK 8

Implement Binary Tree with array and with Array and with linked list

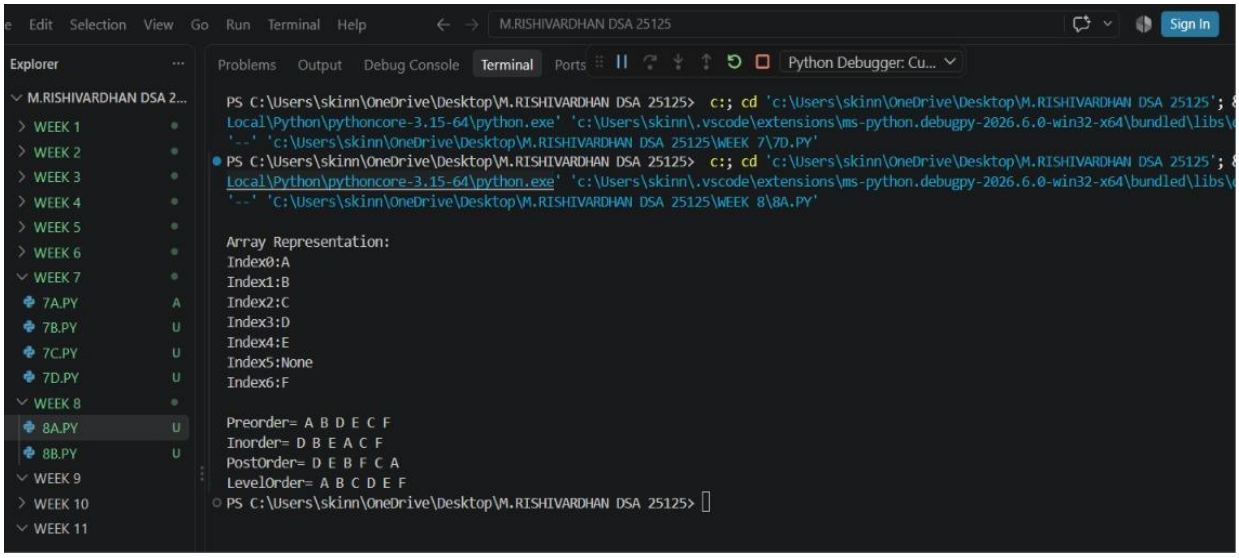
(a) With Linked List



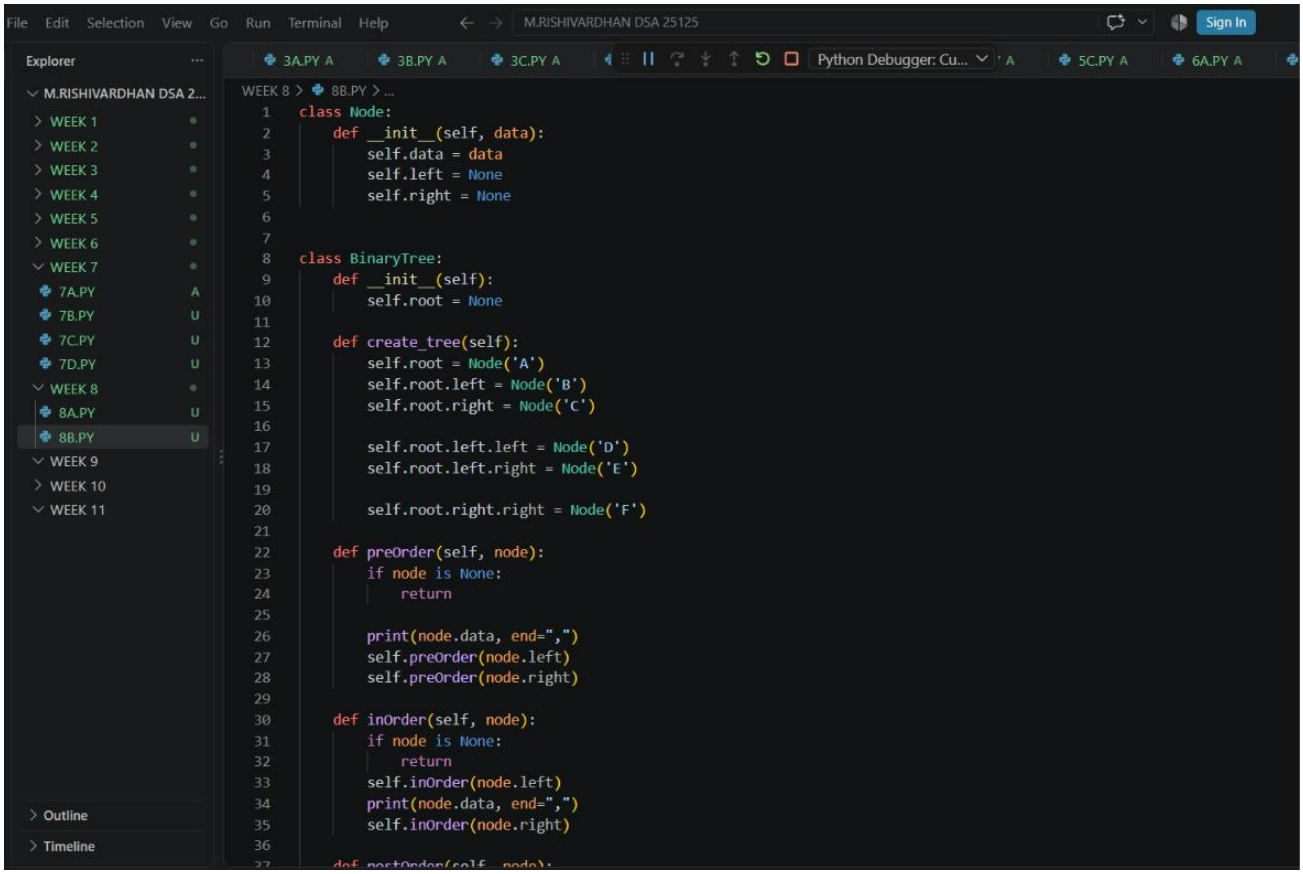

File Edit Selection View Go Run Terminal Help
M.RISHIVARDHAN DSA 25125
Explorer
3C.PY A
4A.PY A
4B.PY A
5A.PY A
5B.PY
Python Debugger: Cu...
M.RISHIVARDHAN DSA 2...
WEEK 1
WEEK 2
WEEK 3
WEEK 4
WEEK 5
WEEK 6
WEEK 7
7A.PY A
7B.PY U
7C.PY U
7D.PY U
WEEK 8
8A.PY U
8B.PY U
WEEK 9
WEEK 10
WEEK 11
Outline
WEEK 8 > 8A.PY > ...
class BinaryTreeArray:
def postorder(self, index=0):
if index >= self.size or self.tree[index] is None:
return
self.postorder(2 * index + 1)
self.postorder(2 * index + 2)
print(self.tree[index], end=" ")
def levelorder(self):
for value in self.tree:
if value is not None:
print(value, end=" ")
def display(self):
print("\nArray Representation:")
for i in range(self.size):
print(f"Index{i}:{self.tree[i]}")
tree = BinaryTreeArray(7)
tree.set_root('A')
tree.set_left(0, 'B')
tree.set_right(0, 'C')
tree.set_left(1, 'D')
tree.set_right(1, 'E')
tree.set_right(2, 'F')
tree.display()
print("\nPreorder=", end=" ")
tree.preorder()
print("\nInorder=", end=" ")
tree.inorder()
print("\nPostOrder=", end=" ")

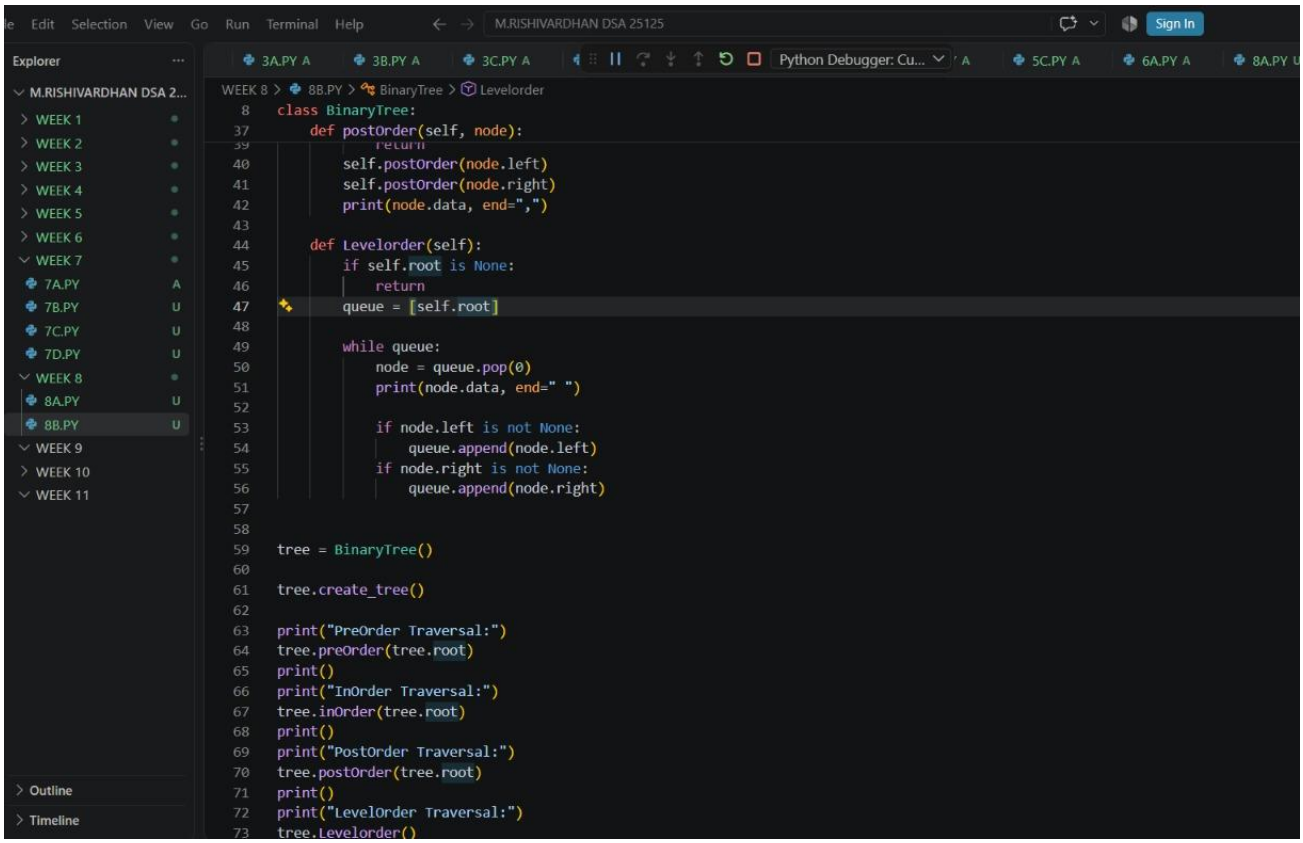
Output:

(a)



(b) With array





```
class BinaryTree:
    def postOrder(self, node):
        self.postOrder(node.left)
        self.postOrder(node.right)
        print(node.data, end=",")

    def Levelorder(self):
        if self.root is None:
            return
        queue = [self.root]

        while queue:
            node = queue.pop(0)
            print(node.data, end=" ")

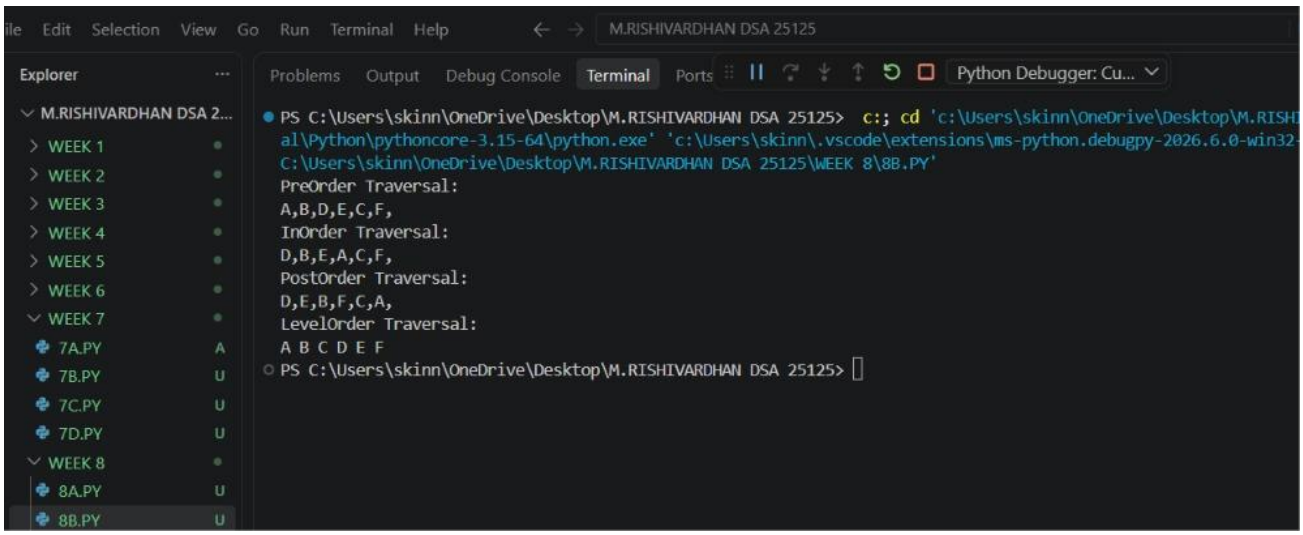
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)


tree = BinaryTree()

tree.create_tree()

print("PreOrder Traversal:")
tree.preOrder(tree.root)
print()
print("InOrder Traversal:")
tree.inOrder(tree.root)
print()
print("PostOrder Traversal:")
tree.postOrder(tree.root)
print()
print("LevelOrder Traversal:")
tree.Levelorder()
```

Output:



```
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125>  c:; cd 'c:\Users\skinn\OneDrive\Desktop\M.RISHI
al\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-
C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 8\8B.PY'
PreOrder Traversal:
A,B,D,E,C,F,
InOrder Traversal:
D,B,E,A,C,F,
PostOrder Traversal:
D,E,B,F,C,A,
LevelOrder Traversal:
A B C D E F
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125>
```

Description:

A Binary Tree is a non-linear data structure in which each node contains data and has at most two children, called the left child and the right child.

This Python program implements a binary tree using a Node class and a BinaryTree class. Each node stores a data element and references to its left and right children. The program creates a binary tree with predefined elements and performs the following traversal operations:

1. Preorder Traversal: Visits the root node, left subtree, and right subtree in that order.
2. Inorder Traversal: Visits the left subtree, root node, and right subtree in that order.
3. Postorder Traversal: Visits the left subtree, right subtree, and root node in that order.
4. Level Order Traversal: Visits the nodes level by level from left to right using a queue.

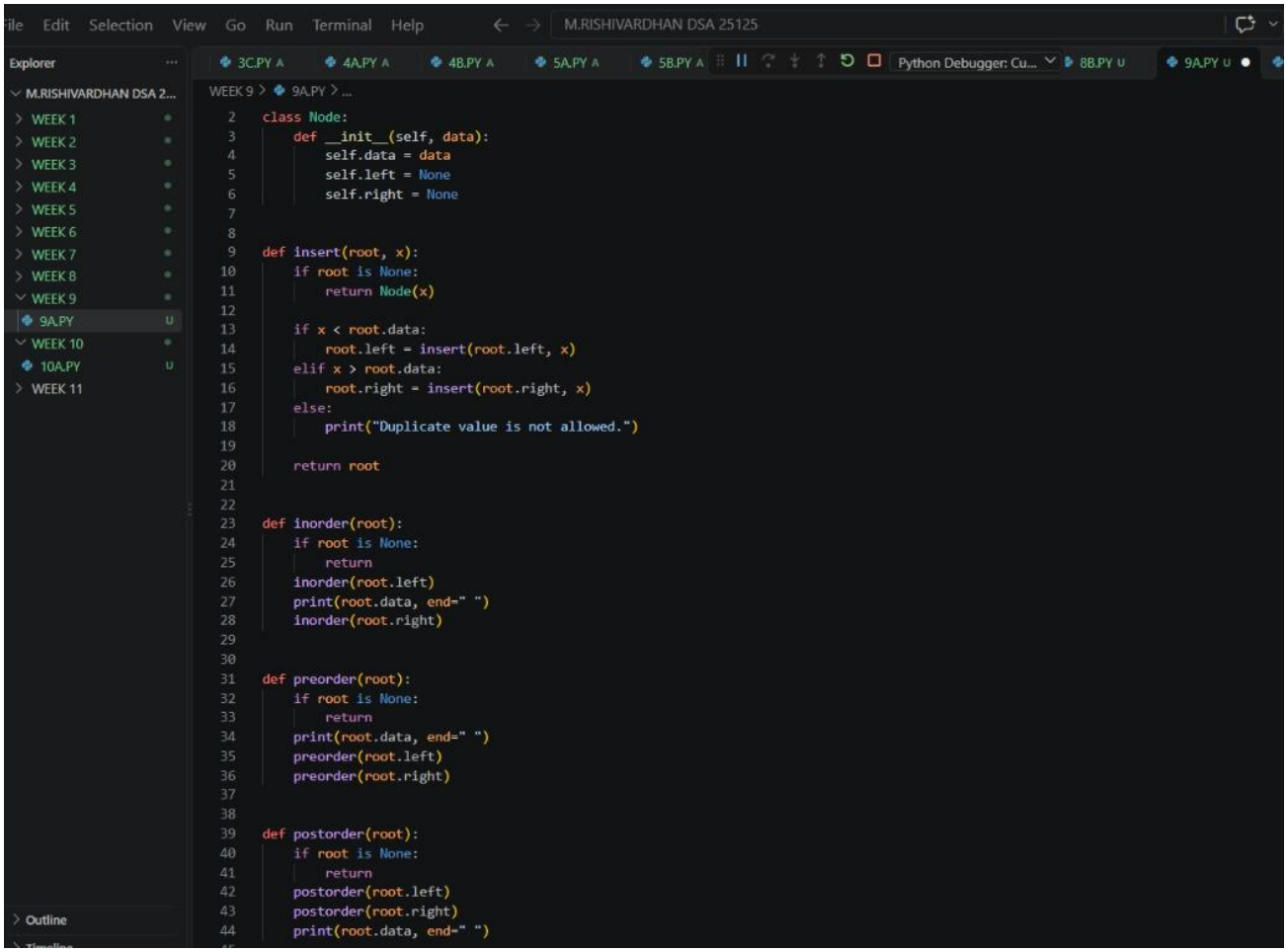
The program creates a binary tree with the elements A, B, C, D, E, and F. The root node is A, B and C are its children, D and E are the children of B, and F is the right child of C.

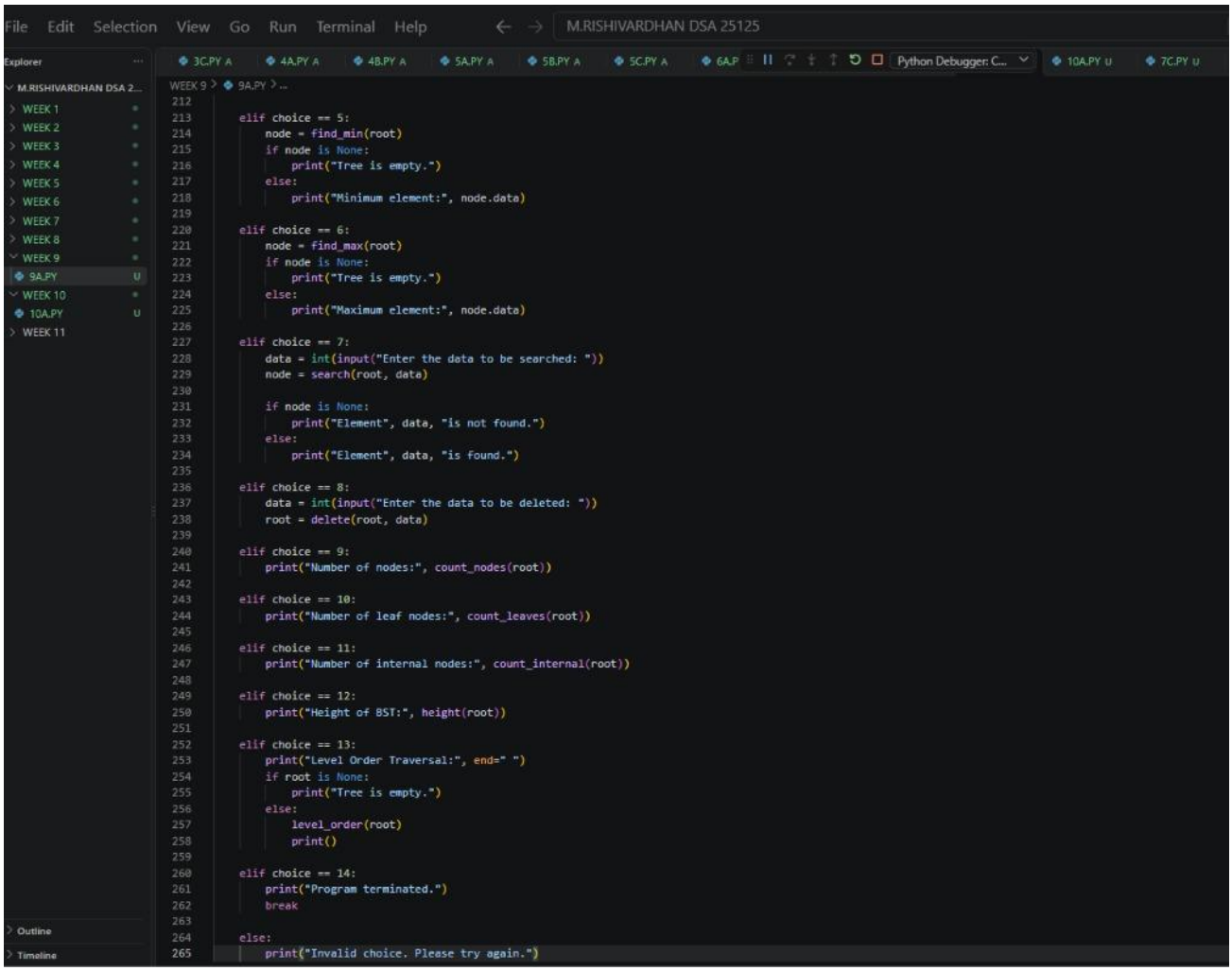
The recursive functions are used to implement preorder, inorder, and postorder traversals. A queue is used in level order traversal to process the nodes level by level.

Result: Thus, the binary tree is created successfully, and preorder, inorder, postorder, and level order traversals are performed and displayed using Python.

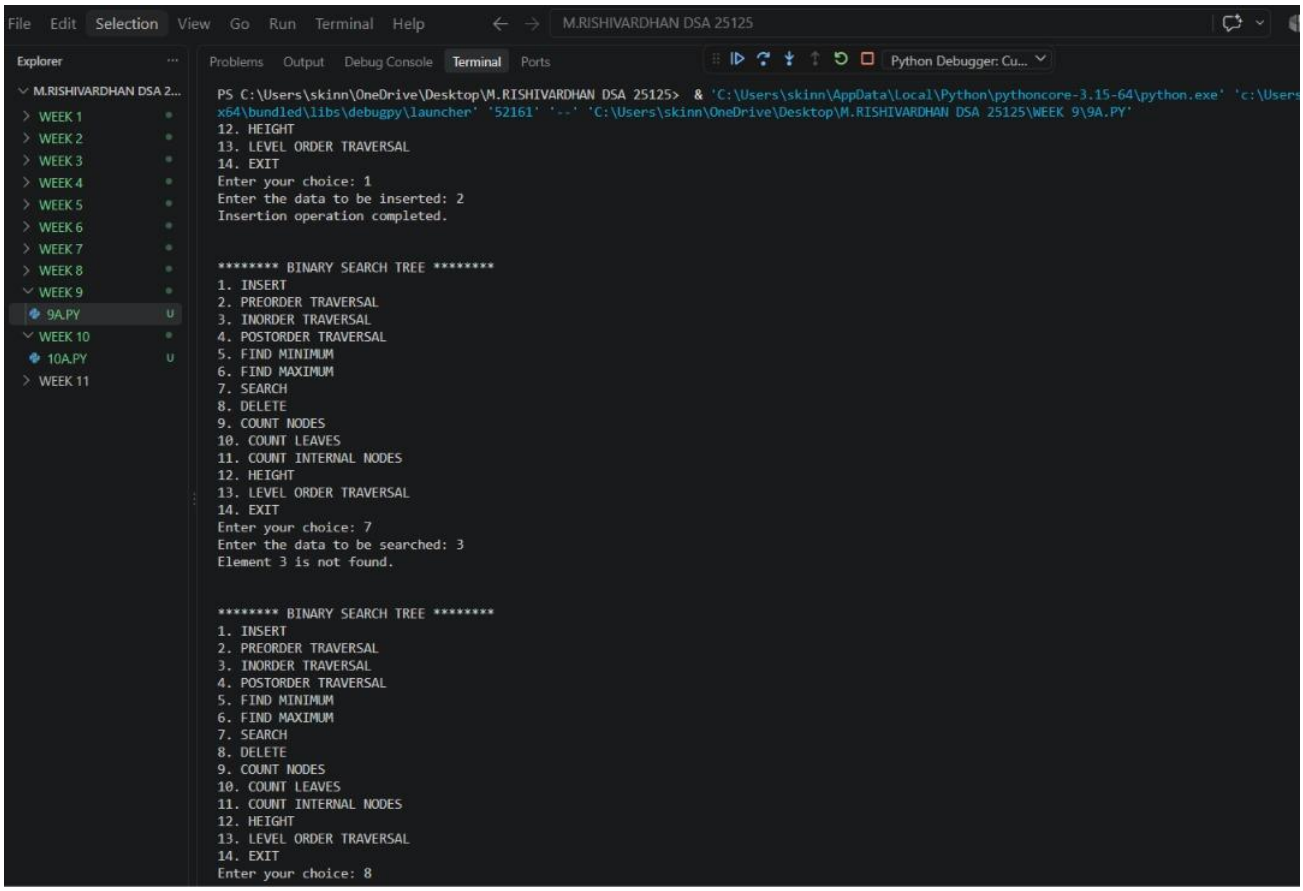
Implement BST and its methods

Program:




File Edit Selection View Go Run Terminal Help
M.RISHIVARDHAN DSA 25125
Explorer
M.RISHIVARDHAN DSA 2...
WEEK 1
WEEK 2
WEEK 3
WEEK 4
WEEK 5
WEEK 6
WEEK 7
WEEK 8
WEEK 9
9A.PY
WEEK 10
10A.PY
WEEK 11
Outline
Timeline
3C.PY A
4A.PY A
4B.PY A
5A.PY A
5B.PY A
5C.PY A
6A.P
Python Debugger: C...
10A.PY U
7C.PY U
WEEK 9 > 9A.PY > ...
212
213 elif choice == 5:
214 node = find_min(root)
215 if node is None:
216 print("Tree is empty.")
217 else:
218 print("Minimum element:", node.data)
219
220 elif choice == 6:
221 node = find_max(root)
222 if node is None:
223 print("Tree is empty.")
224 else:
225 print("Maximum element:", node.data)
226
227 elif choice == 7:
228 data = int(input("Enter the data to be searched: "))
229 node = search(root, data)
230
231 if node is None:
232 print("Element", data, "is not found.")
233 else:
234 print("Element", data, "is found.")
235
236 elif choice == 8:
237 data = int(input("Enter the data to be deleted: "))
238 root = delete(root, data)
239
240 elif choice == 9:
241 print("Number of nodes:", count_nodes(root))
242
243 elif choice == 10:
244 print("Number of leaf nodes:", count_leaves(root))
245
246 elif choice == 11:
247 print("Number of internal nodes:", count_internal(root))
248
249 elif choice == 12:
250 print("Height of BST:", height(root))
251
252 elif choice == 13:
253 print("Level Order Traversal:", end=" ")
254 if root is None:
255 print("Tree is empty.")
256 else:
257 level_order(root)
258 print()
259
260 elif choice == 14:
261 print("Program terminated.")
262 break
263
264 else:
265 print("Invalid choice. Please try again.")

Output:



Description:

A Binary Search Tree (BST) is a binary tree in which each node contains a data value, a left pointer, and a right pointer. The left subtree contains values smaller than the root node, and the right subtree contains values greater than the root node.

This Python program implements a Binary Search Tree using a class and recursive functions. It provides a menu-driven interface to perform the following operations:

1. Insertion: Inserts a new element into the BST while maintaining the BST property. Duplicate values are not allowed.
2. Preorder Traversal: Visits the root node, left subtree, and right subtree.
3. Inorder Traversal: Visits the left subtree, root node, and right subtree. It displays the elements in ascending order.
4. Postorder Traversal: Visits the left subtree, right subtree, and root node.
5. Find Minimum: Finds the minimum element by traversing the leftmost nodes of the tree.
6. Find Maximum: Finds the maximum element by traversing the rightmost nodes of the tree.
7. Search: Searches for a specified element in the BST using comparisons.
8. Deletion: Deletes an element from the BST while maintaining its structure. It handles nodes with no children, one child, or two children.
9. Count Nodes: Calculates the total number of nodes in the tree.
10. Count Leaf Nodes: Counts the nodes that do not have any children.
11. Count Internal Nodes: Counts the nodes that have at least one child, excluding leaf nodes.
12. Find Height: Calculates the height of the BST, defined as the number of edges in the longest

path from the root to a leaf. An empty tree has a height of -1.

13. Level Order Traversal: Displays the nodes level by level, starting from the root.
14. Exit: Terminates the program.

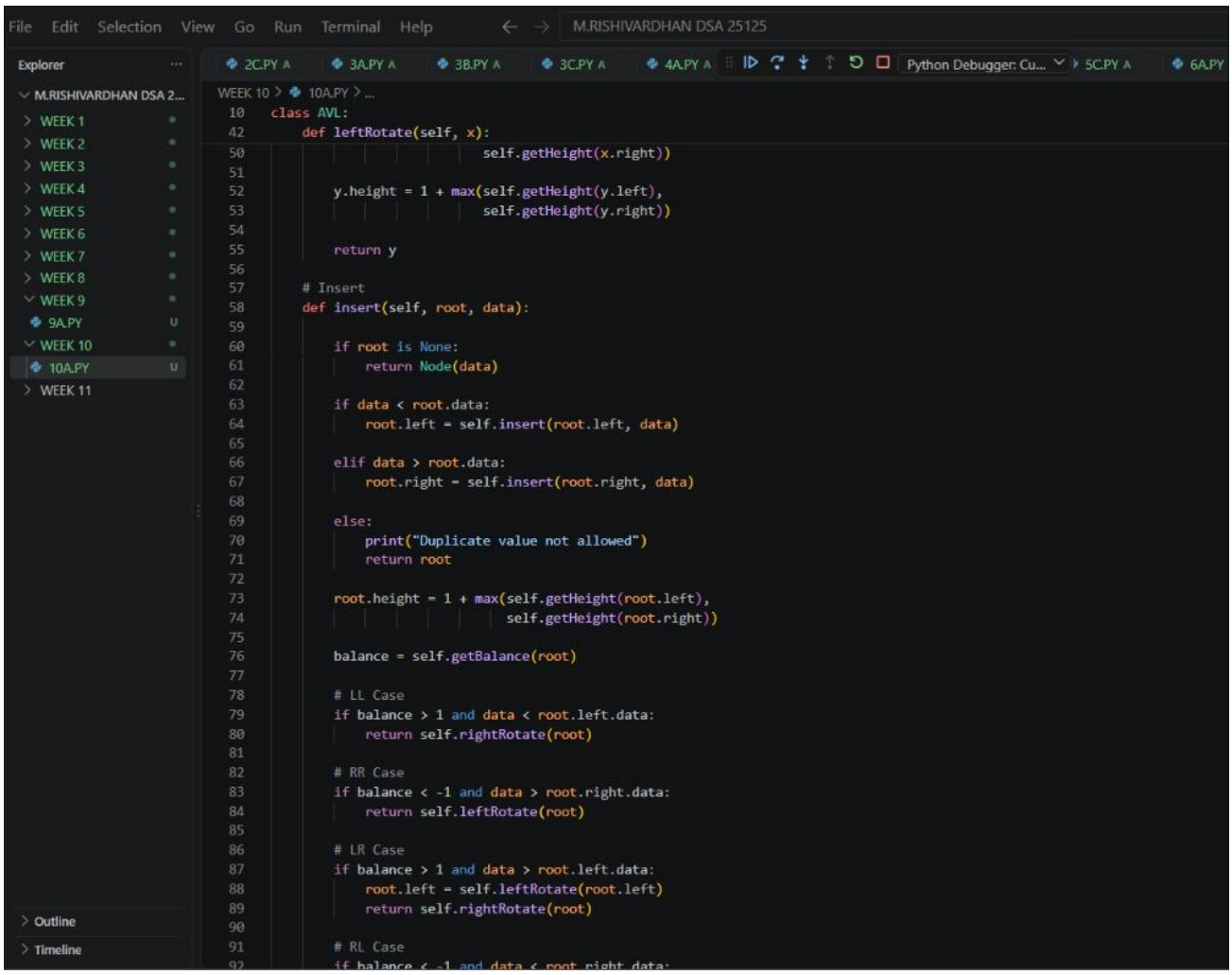
The program uses recursion for insertion, searching, deletion, counting, and tree traversals. It continues displaying the menu until the user chooses the exit option.

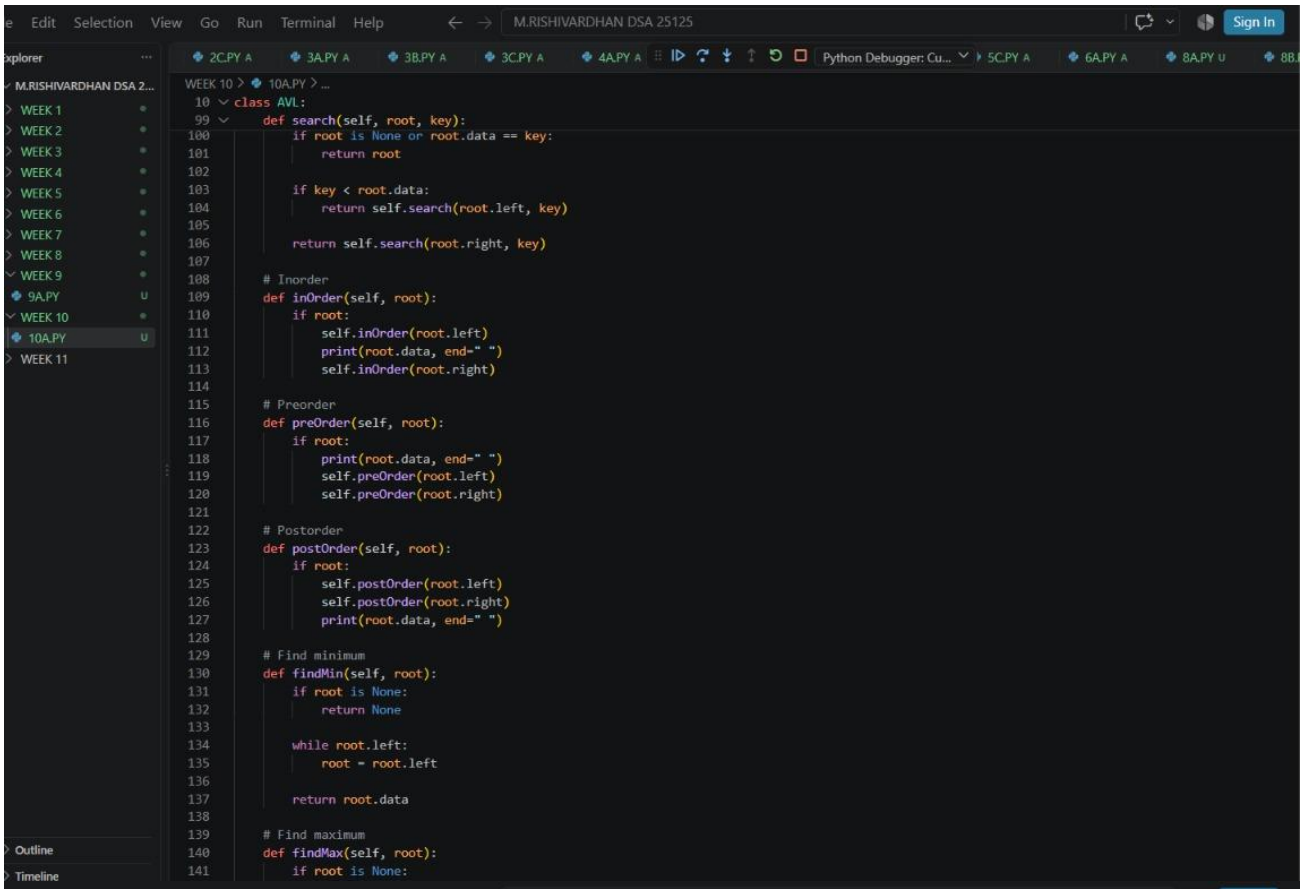
Result: Thus, the Binary Search Tree is implemented successfully in Python, supporting insertion, deletion, searching, traversal, counting, and height calculation operations.

WEEK 10

Write a python program to implement AVL trees and all its methods

Input:





```
class AVL:
    def search(self, root, key):
        if root is None or root.data == key:
            return root

        if key < root.data:
            return self.search(root.left, key)

        return self.search(root.right, key)

    # Inorder
    def inOrder(self, root):
        if root:
            self.inOrder(root.left)
            print(root.data, end=" ")
            self.inOrder(root.right)

    # Preorder
    def preOrder(self, root):
        if root:
            print(root.data, end=" ")
            self.preOrder(root.left)
            self.preOrder(root.right)

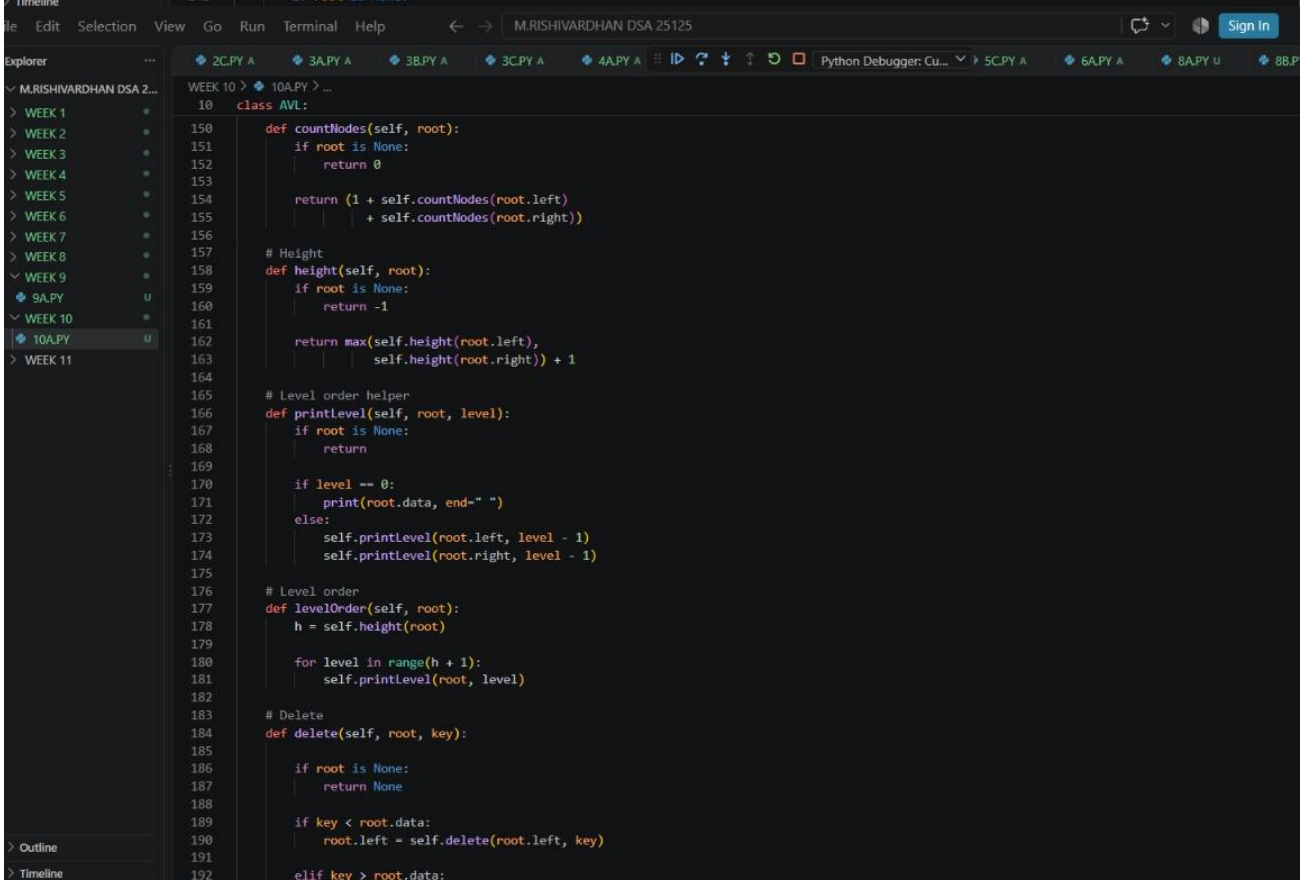
    # Postorder
    def postOrder(self, root):
        if root:
            self.postOrder(root.left)
            self.postOrder(root.right)
            print(root.data, end=" ")

    # Find minimum
    def findMin(self, root):
        if root is None:
            return None

        while root.left:
            root = root.left

        return root.data

    # Find maximum
    def findMax(self, root):
        if root is None:
```



```
class AVL:
    def countNodes(self, root):
        if root is None:
            return 0

        return (1 + self.countNodes(root.left)
                  + self.countNodes(root.right))

    # Height
    def height(self, root):
        if root is None:
            return -1

        return max(self.height(root.left),
                   self.height(root.right)) + 1

    # Level order helper
    def printLevel(self, root, level):
        if root is None:
            return

        if level == 0:
            print(root.data, end=" ")
        else:
            self.printLevel(root.left, level - 1)
            self.printLevel(root.right, level - 1)

    # Level order
    def levelOrder(self, root):
        h = self.height(root)

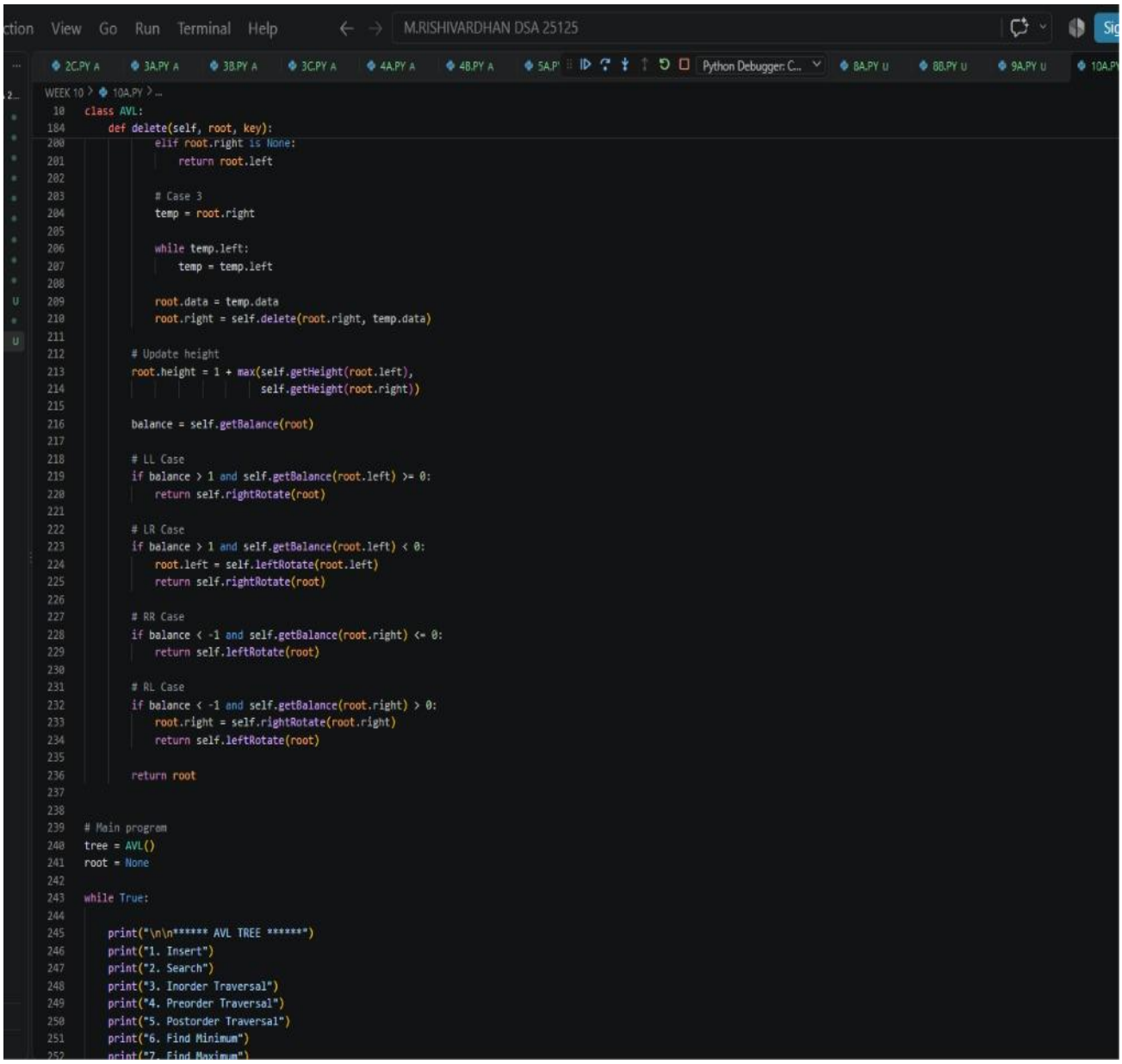
        for level in range(h + 1):
            self.printLevel(root, level)

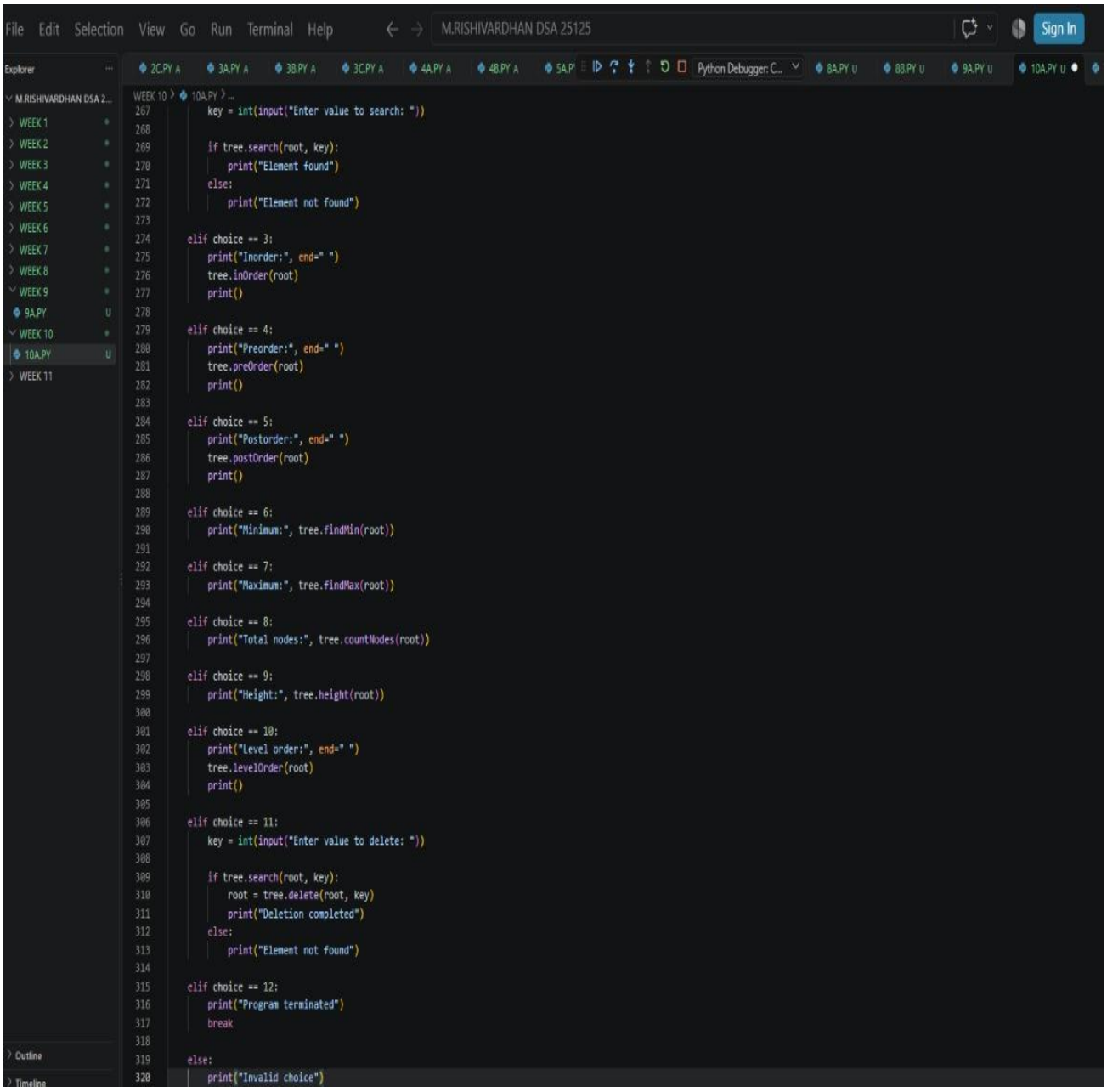
    # Delete
    def delete(self, root, key):

        if root is None:
            return None

        if key < root.data:
            root.left = self.delete(root.left, key)

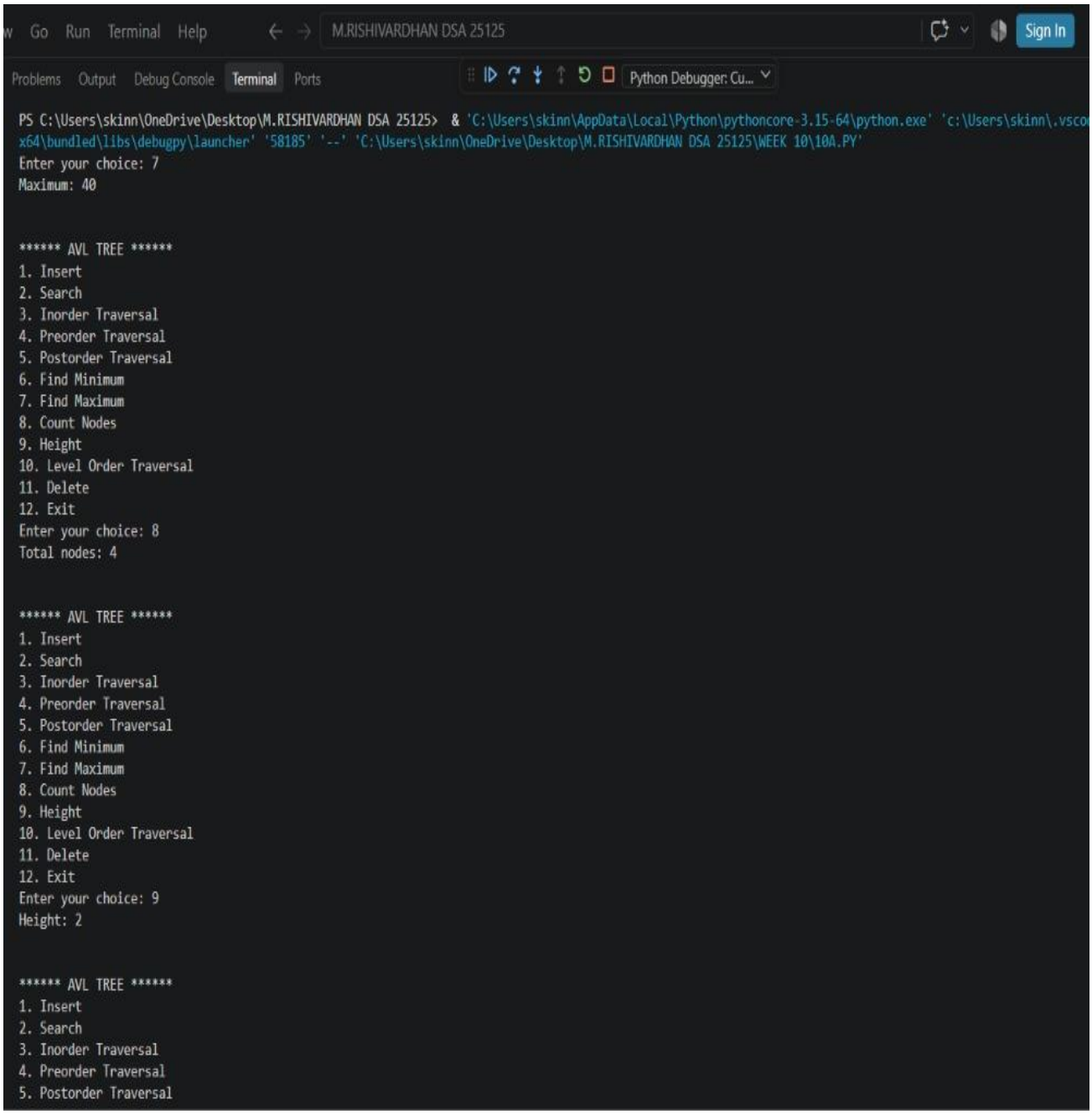
        elif key > root.data:
```

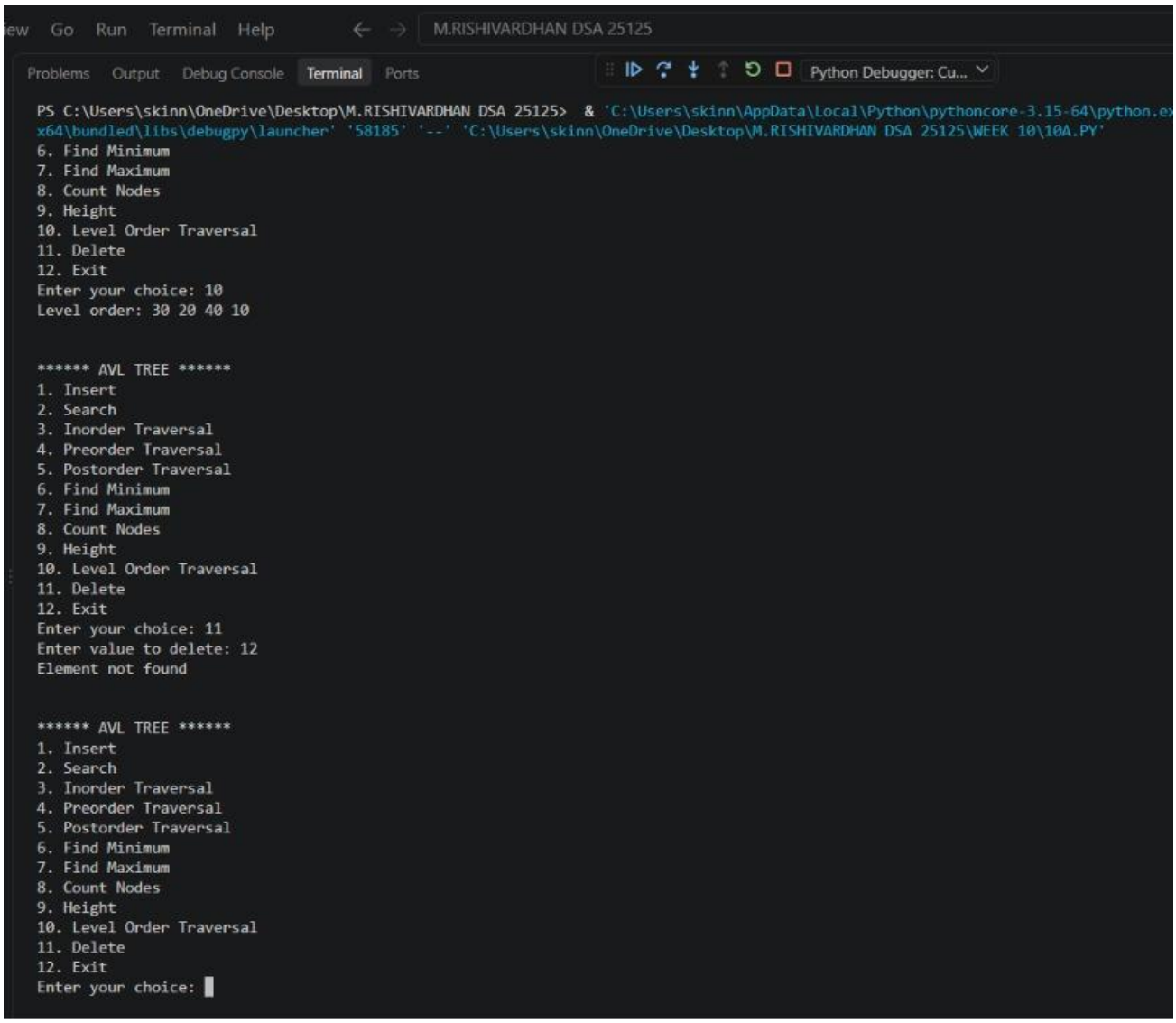

ction View Go Run Terminal Help
M.RISHIVARDHAN DSA 25125
2C.PY A
3A.PY A
3B.PY A
3C.PY A
4A.PY A
4B.PY A
5A.P
Python Debugger: C...
8A.PY U
8B.PY U
9A.PY U
10A.PY
WEEK 10 > 10A.PY > ...
10
class AVL:
184
def delete(self, root, key):
200
elif root.right is None:
201
return root.left
202
203
Case 3
204
temp = root.right
205
206
while temp.left:
207
temp = temp.left
208
209
root.data = temp.data
210
root.right = self.delete(root.right, temp.data)
211
212
Update height
213
root.height = 1 + max(self.getHeight(root.left),
214
self.getHeight(root.right))
215
216
balance = self.getBalance(root)
217
218
LL Case
219
if balance > 1 and self.getBalance(root.left) >= 0:
220
return self.rightRotate(root)
221
222
LR Case
223
if balance > 1 and self.getBalance(root.left) < 0:
224
root.left = self.leftRotate(root.left)
225
return self.rightRotate(root)
226
227
RR Case
228
if balance < -1 and self.getBalance(root.right) <= 0:
229
return self.leftRotate(root)
230
231
RL Case
232
if balance < -1 and self.getBalance(root.right) > 0:
233
root.right = self.rightRotate(root.right)
234
return self.leftRotate(root)
235
236
return root
237
238
239
Main program
240
tree = AVL()
241
root = None
242
243
while True:
244
245
print("\n\n****** AVL TREE *******")
246
print("1. Insert")
247
print("2. Search")
248
print("3. Inorder Traversal")
249
print("4. Preorder Traversal")
250
print("5. Postorder Traversal")
251
print("6. Find Minimum")
252
print("7. Find Maximum")


File Edit Selection View Go Run Terminal Help
M.RISHIVARDHAN DSA 25125
Sign In
Explorer
2C.PY A
3A.PY A
3B.PY A
3C.PY A
4A.PY A
4B.PY A
Python Debugger: C...
8A.PY U
8B.PY U
9A.PY U
10A.PY U
M.RISHIVARDHAN DSA 2...
WEEK 1
WEEK 2
WEEK 3
WEEK 4
WEEK 5
WEEK 6
WEEK 7
WEEK 8
WEEK 9
9A.PY U
WEEK 10
10A.PY U
WEEK 11
Outline
Timeline
WEEK 10 > 10A.PY > ...
key = int(input("Enter value to search: "))
if tree.search(root, key):
print("Element found")
else:
print("Element not found")
elif choice == 3:
print("Inorder:", end=" ")
tree.inOrder(root)
print()
elif choice == 4:
print("Preorder:", end=" ")
tree.preOrder(root)
print()
elif choice == 5:
print("Postorder:", end=" ")
tree.postOrder(root)
print()
elif choice == 6:
print("Minimum:", tree.findMin(root))
elif choice == 7:
print("Maximum:", tree.findMax(root))
elif choice == 8:
print("Total nodes:", tree.countNodes(root))
elif choice == 9:
print("Height:", tree.height(root))
elif choice == 10:
print("Level order:", end=" ")
tree.levelOrder(root)
print()
elif choice == 11:
key = int(input("Enter value to delete: "))
if tree.search(root, key):
root = tree.delete(root, key)
print("Deletion completed")
else:
print("Element not found")
elif choice == 12:
print("Program terminated")
break
else:
print("Invalid choice")


w Go Run Terminal Help
M.RISHIVARDHAN DSA 25125
Sign In
Problems Output Debug Console Terminal Ports
Python Debugger: Cu...
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125> & 'C:\Users\skinn\AppData\Local\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vsco
x64\bundled\libs\debugpy\launcher' '58185' '--' 'C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 10\10A.PY'
****** AVL TREE ******
1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Find Minimum
7. Find Maximum
8. Count Nodes
9. Height
10. Level Order Traversal
11. Delete
12. Exit
Enter your choice: 1
Enter value: 40
Insertion completed
****** AVL TREE ******
1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Find Minimum
7. Find Maximum
8. Count Nodes
9. Height
10. Level Order Traversal
11. Delete
12. Exit
Enter your choice: 1
Enter value: 20
Insertion completed
****** AVL TREE ******
1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Find Minimum


ew Go Run Terminal Help
M.RISHIVARDHAN DSA 25125
Sign In
Problems Output Debug Console Terminal Ports
Python Debugger: Cu...
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125> & 'C:\Users\skinn\AppData\Local\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vscode\e
x64\bundled\libs\debugpy\launcher' '58185' '--' 'C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 10\10A.PY'
5. Postorder Traversal
6. Find Minimum
7. Find Maximum
8. Count Nodes
9. Height
10. Level Order Traversal
11. Delete
12. Exit
Enter your choice: 4
Preorder: 30 20 10 40
****** AVL TREE ******
1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Find Minimum
7. Find Maximum
8. Count Nodes
9. Height
10. Level Order Traversal
11. Delete
12. Exit
Enter your choice: 3
Inorder: 10 20 30 40
****** AVL TREE ******
1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Find Minimum
7. Find Maximum
8. Count Nodes
9. Height
10. Level Order Traversal
11. Delete
12. Exit
Enter your choice: 5
Postorder: 10 20 40 30


w Go Run Terminal Help
M.RISHIVARDHAN DSA 25125
Sign In
Problems Output Debug Console Terminal Ports
Python Debugger: Cu...
PS C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125> & 'C:\Users\skinn\AppData\Local\Python\pythoncore-3.15-64\python.exe' 'c:\Users\skinn\.vsco
x64\bundled\libs\debugpy\launcher' '58185' '--' 'C:\Users\skinn\OneDrive\Desktop\M.RISHIVARDHAN DSA 25125\WEEK 10\10A.PY'
Enter your choice: 7
Maximum: 40
****** AVL TREE ******
1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Find Minimum
7. Find Maximum
8. Count Nodes
9. Height
10. Level Order Traversal
11. Delete
12. Exit
Enter your choice: 8
Total nodes: 4
****** AVL TREE ******
1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Find Minimum
7. Find Maximum
8. Count Nodes
9. Height
10. Level Order Traversal
11. Delete
12. Exit
Enter your choice: 9
Height: 2
****** AVL TREE ******
1. Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal

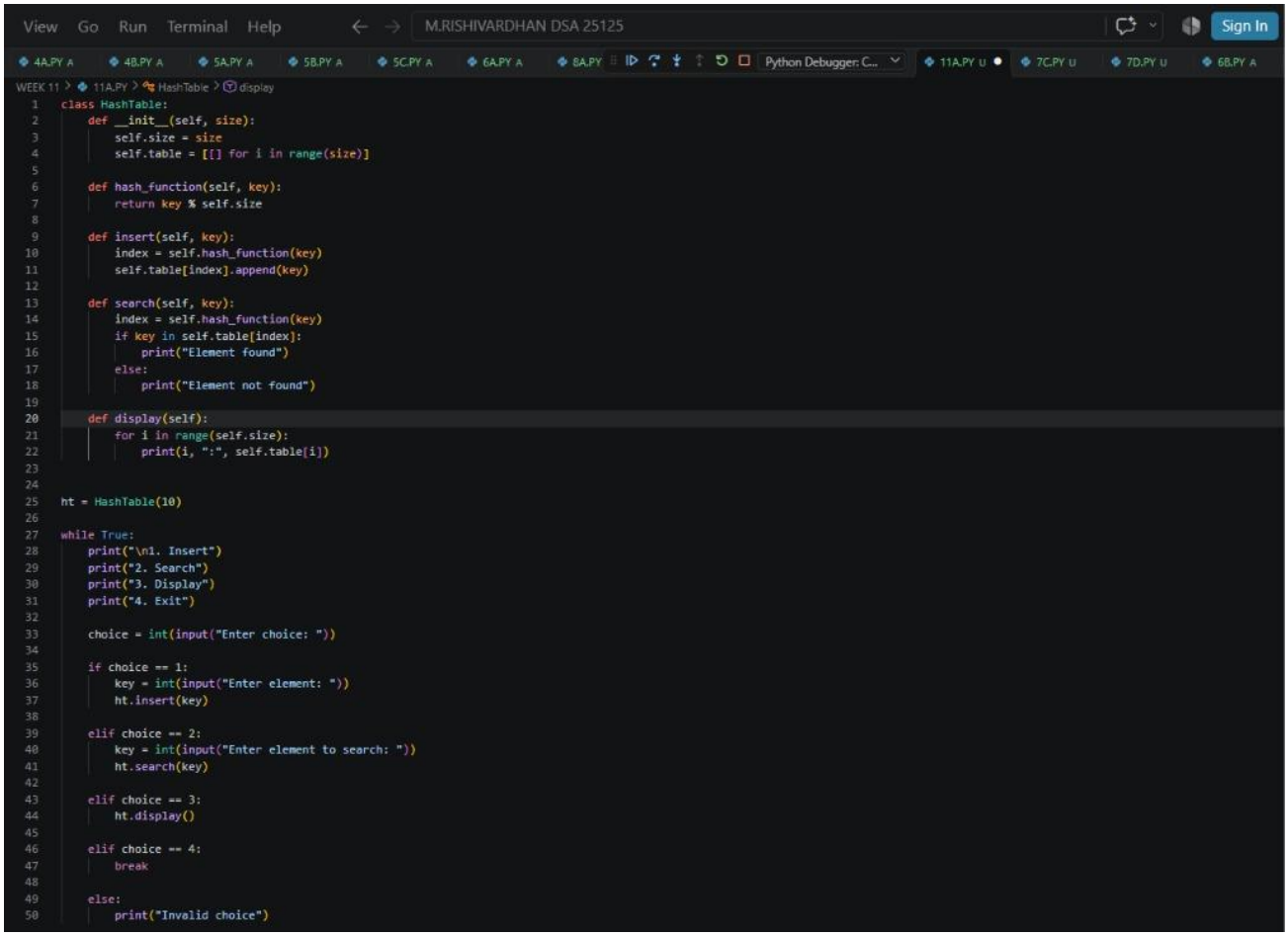


Description:

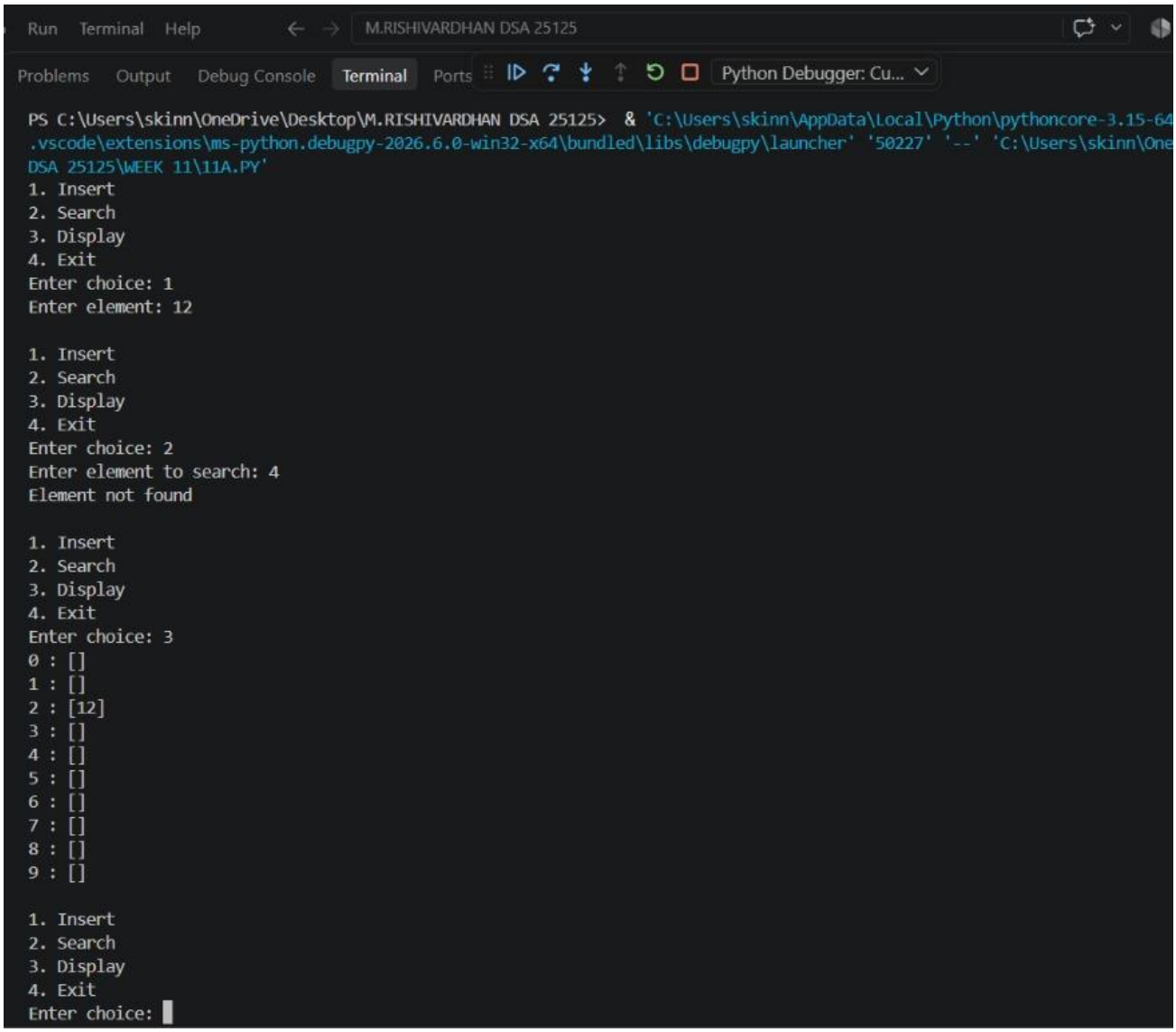
An AVL Tree is a self-balancing Binary Search Tree (BST).
It maintains balance after every insertion and deletion.
Balance Factor (BF) = Height of Left Subtree − Height of Right Subtree.
The balance factor of each node must be -1, 0 or +1.
AVL Trees use rotations to maintain balance.
There are four rotations: LL, RR, LR, and RL.
Searching, insertion and deletion take O (log n) time.
AVL Trees provide efficient search and data management.

Write a python program to implement hashing concept

Input:



Output:



Description:

Hashing is a technique used to store and retrieve data quickly.
It uses a hash function to convert a key into an index in a hash table.
A hash table stores elements at the calculated index.
A collision occurs when two keys generate the same index.
Collision can be handled using techniques like chaining and linear probing.
In chaining, multiple elements are stored at the same index using a list.
Hashing provides an average time complexity of O(1) for insertion and searching.
It is widely used in databases, dictionaries and data storage.